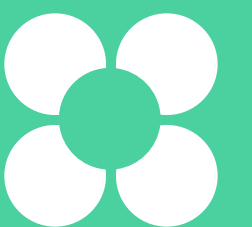


Выделение признаков и поиск похожих изображений

Егор Конягин

Сотрудник лаборатории высокопроизводительных
вычислений и больших данных Сколтеха



Егор Конягин

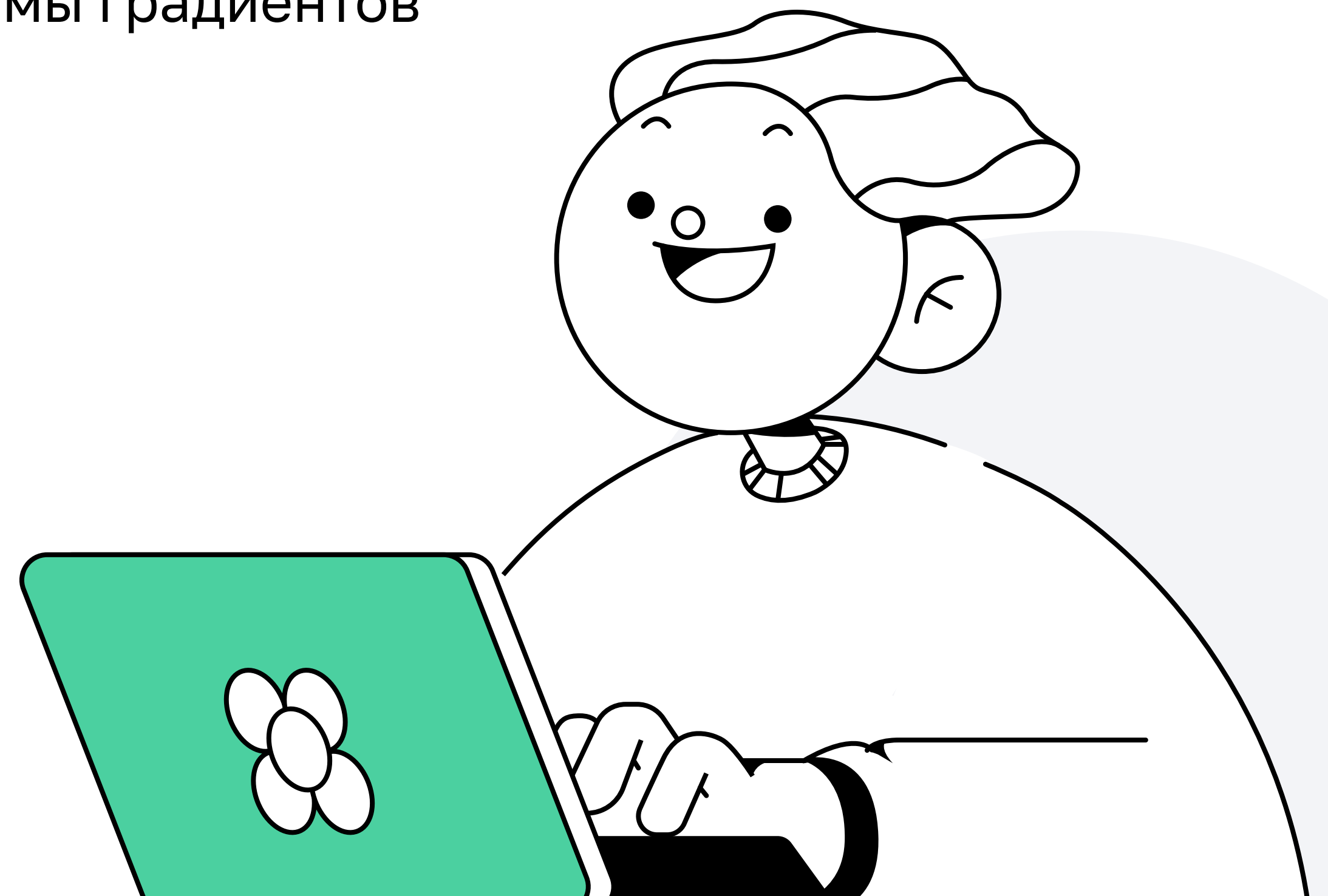
О спикере:

- Сотрудник лаборатории высокопроизводительных вычислений и больших данных, Сколтех
- Разрабатывал библиотеку Sberbank LightAutoML
- Разрабатывал систему машинного зрения для аэропорта Шереметьево

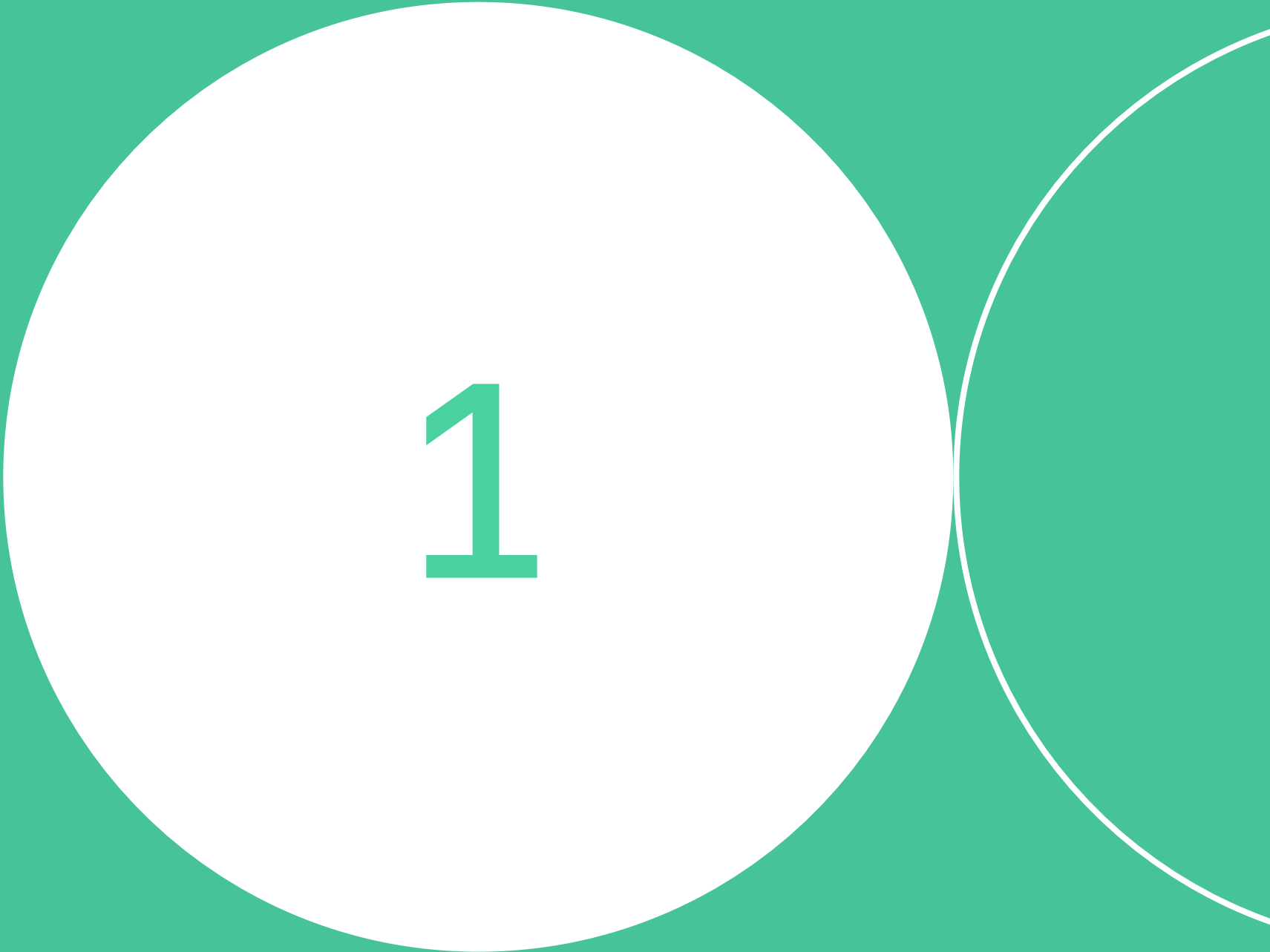


План занятия

- 1 Примеры задач компьютерного зрения
- 2 Анализ главных компонент в задачах CV
- 3 Выделение признаков изображений. Гистограммы цветов
- 4 Выделение признаков изображений. Гистограммы градиентов
- 5 Характерные точки
- 6 Нейронные сети в компьютерном зрении



Примеры задач компьютерного зрения



1

Распознавание лиц

Предсказанный ответ: Пауэлл.
Истинный ответ: Пауэлл



Предсказанный ответ: Буш.
Истинный ответ: Буш



Предсказанный ответ: Чавес.
Истинный ответ: Чавес



Предсказанный ответ: Буш.
Истинный ответ: Буш



Предсказанный ответ: Рамсфелд.
Истинный ответ: Рамсфелд



Предсказанный ответ: Блэр.
Истинный ответ: Шрёдер



Предсказанный ответ: Шарон.
Истинный ответ: Шарон

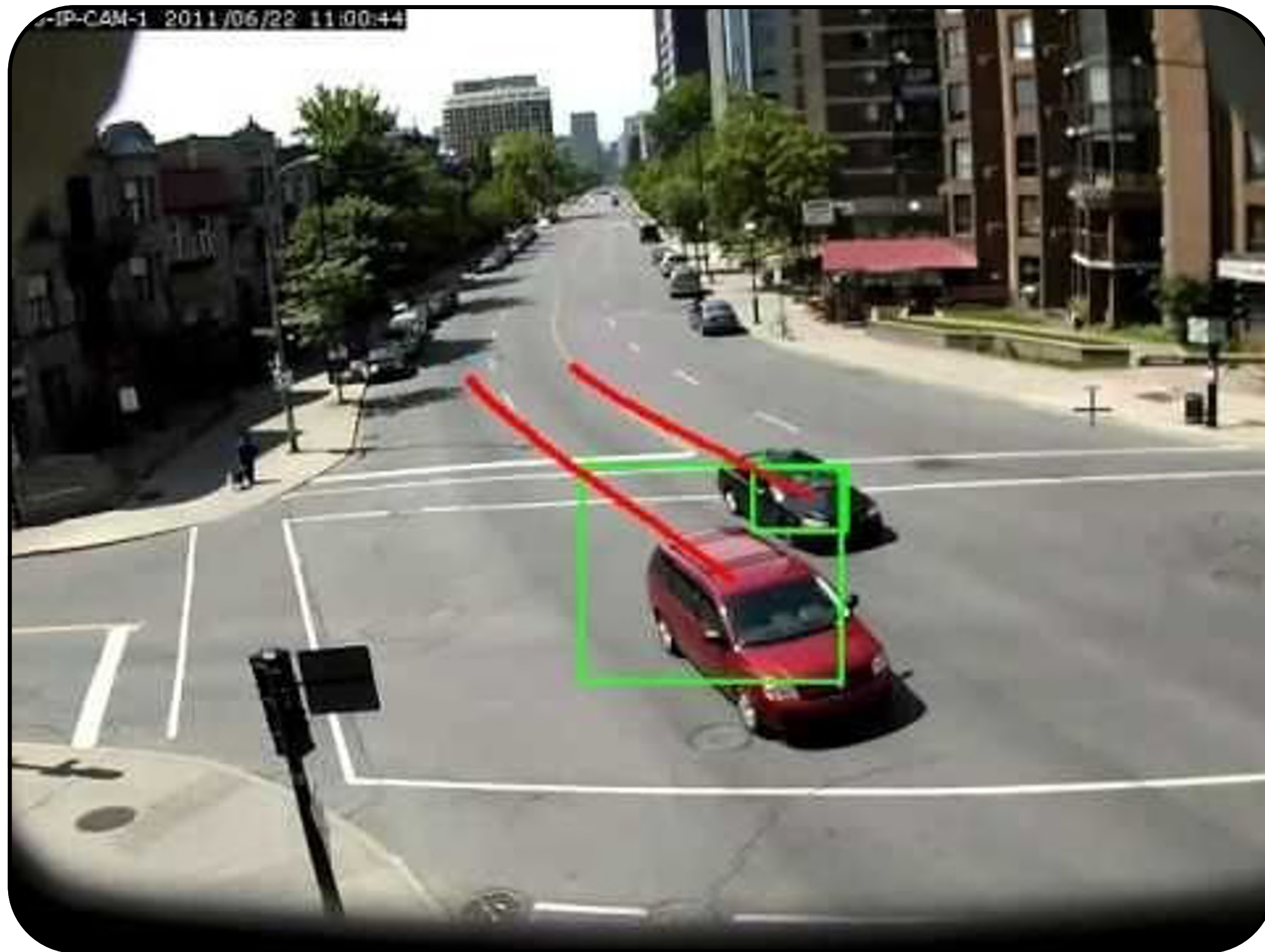


Предсказанный ответ: Шрёдер.
Истинный ответ: Шрёдер



Пример: optical flow

Слежение за несколькими объектами с использованием оптического потока



Пример: поиск похожих изображений (CBIR)

Искомое изображение



Выданные результаты



Анализ главных компонент в задачах CV



2

PCA – principal components analysis

PCA (principal components analysis) – анализ главных компонент:

- изображение можно представить в виде вектора длины $H \times W$
- большая размерность данных (число пикселей) затрудняет их обработку
- чтобы сократить размерность, применяется метод PCA

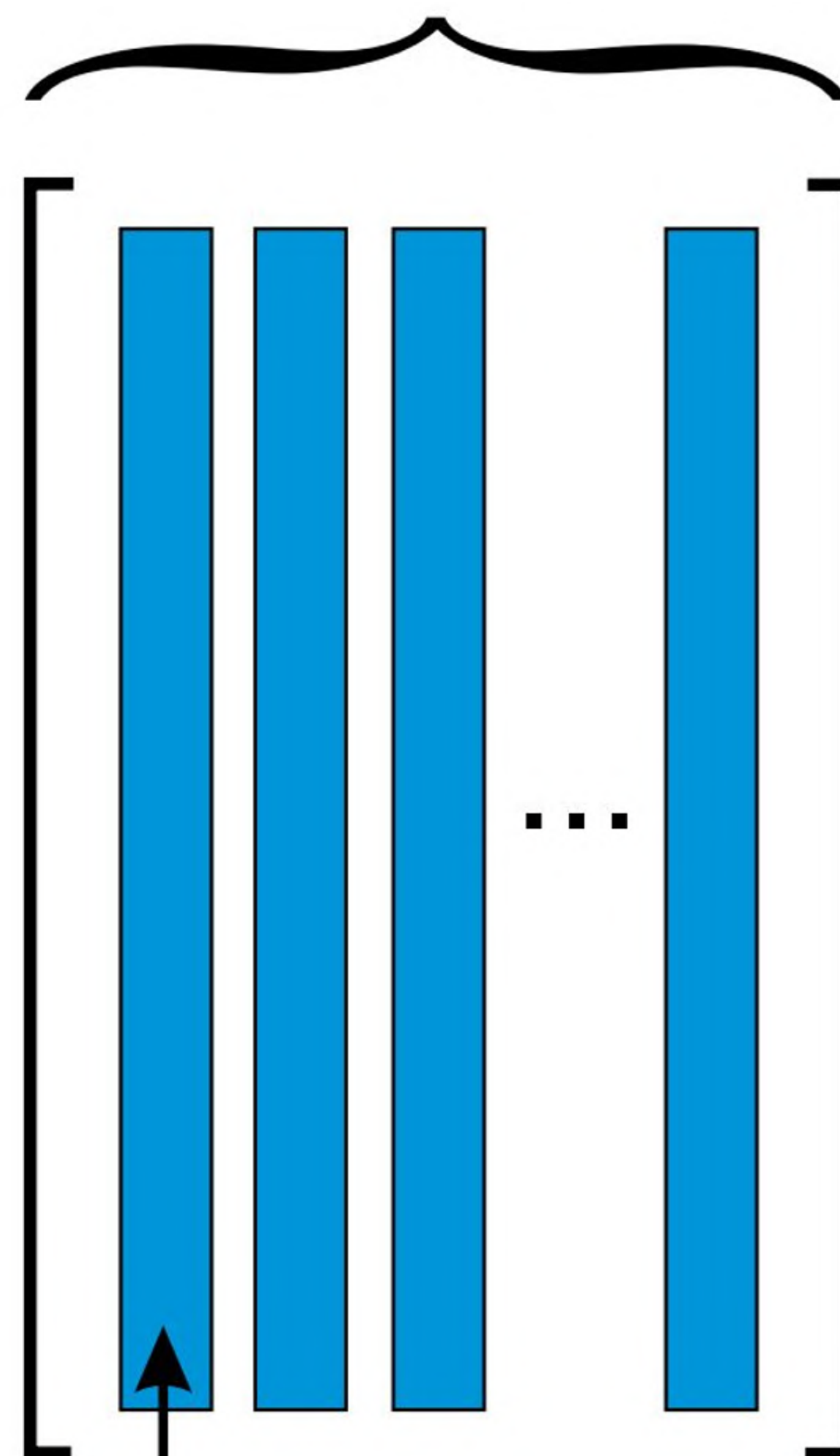
PCA – principal components analysis

32 экземпляра исходного изображения:



PCA – principal components analysis

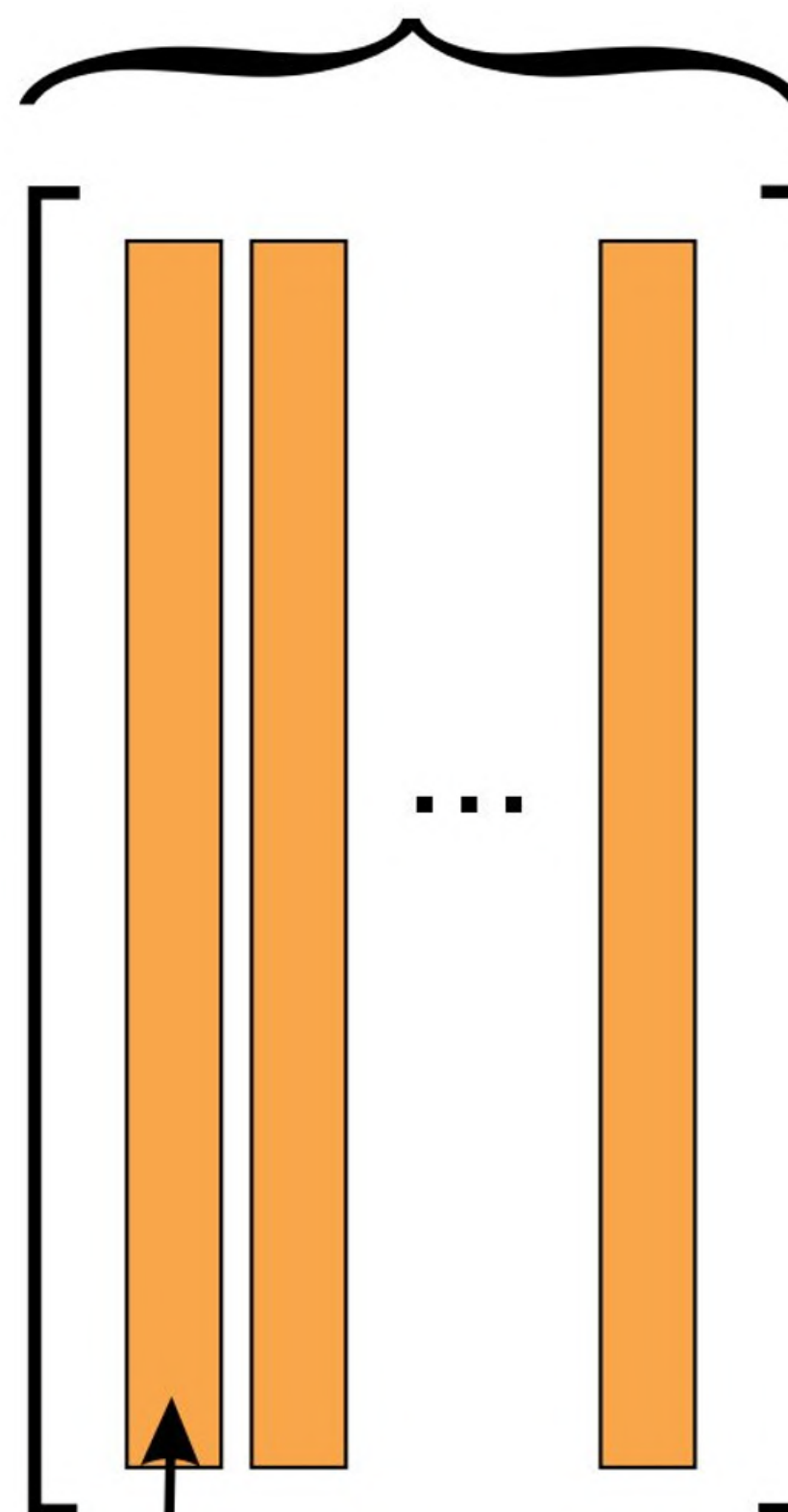
Матрица данных N
наблюдаемых изображений



Одно изображение

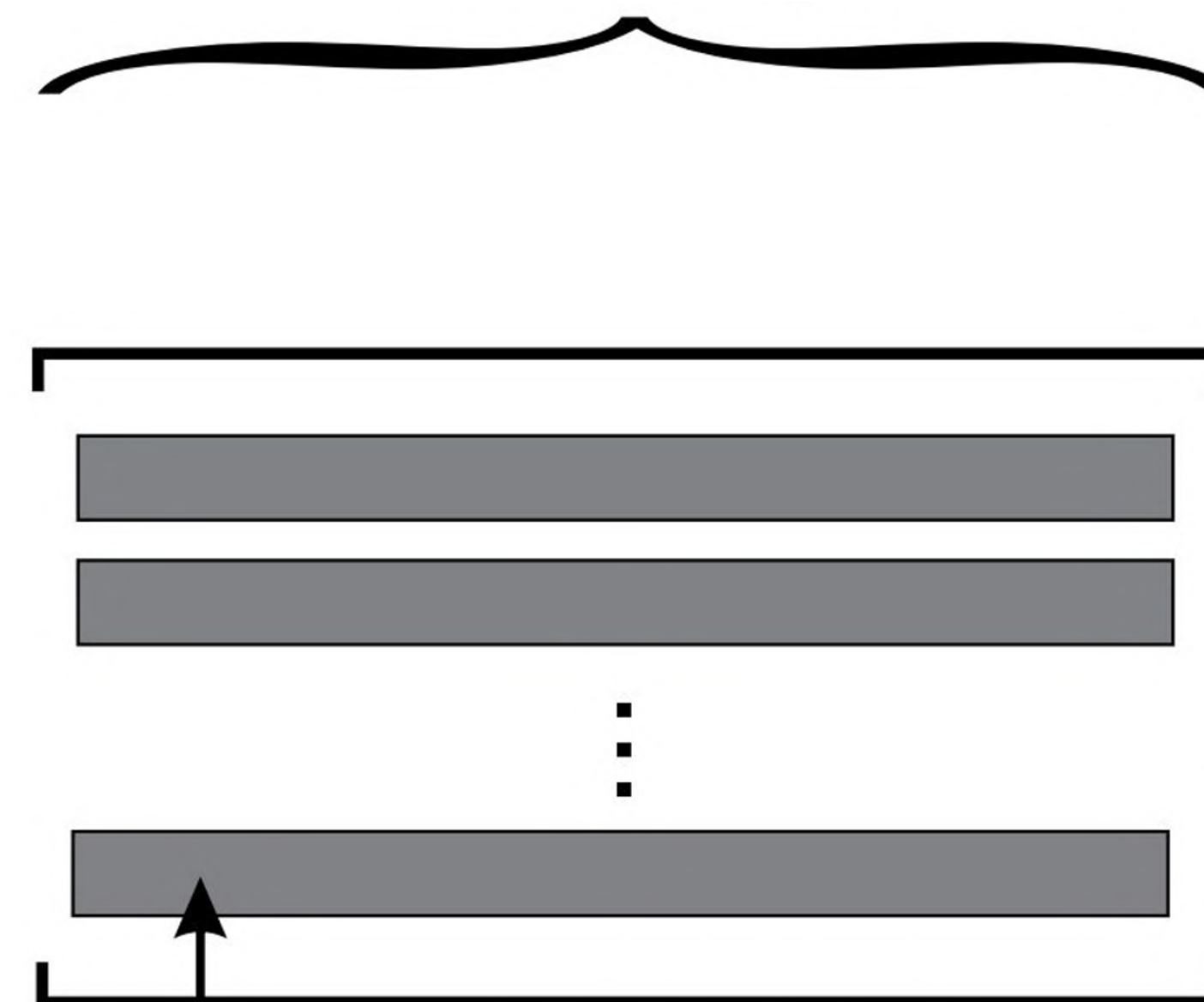
$\approx$

L базисные векторы



Один базисный вектор

PCA-представление N изображений



Одно изображение,
представленное PCA

PCA – principal components analysis

Восстановление изображения по четырём базисным векторам, которые могут отображаться в виде изображений:

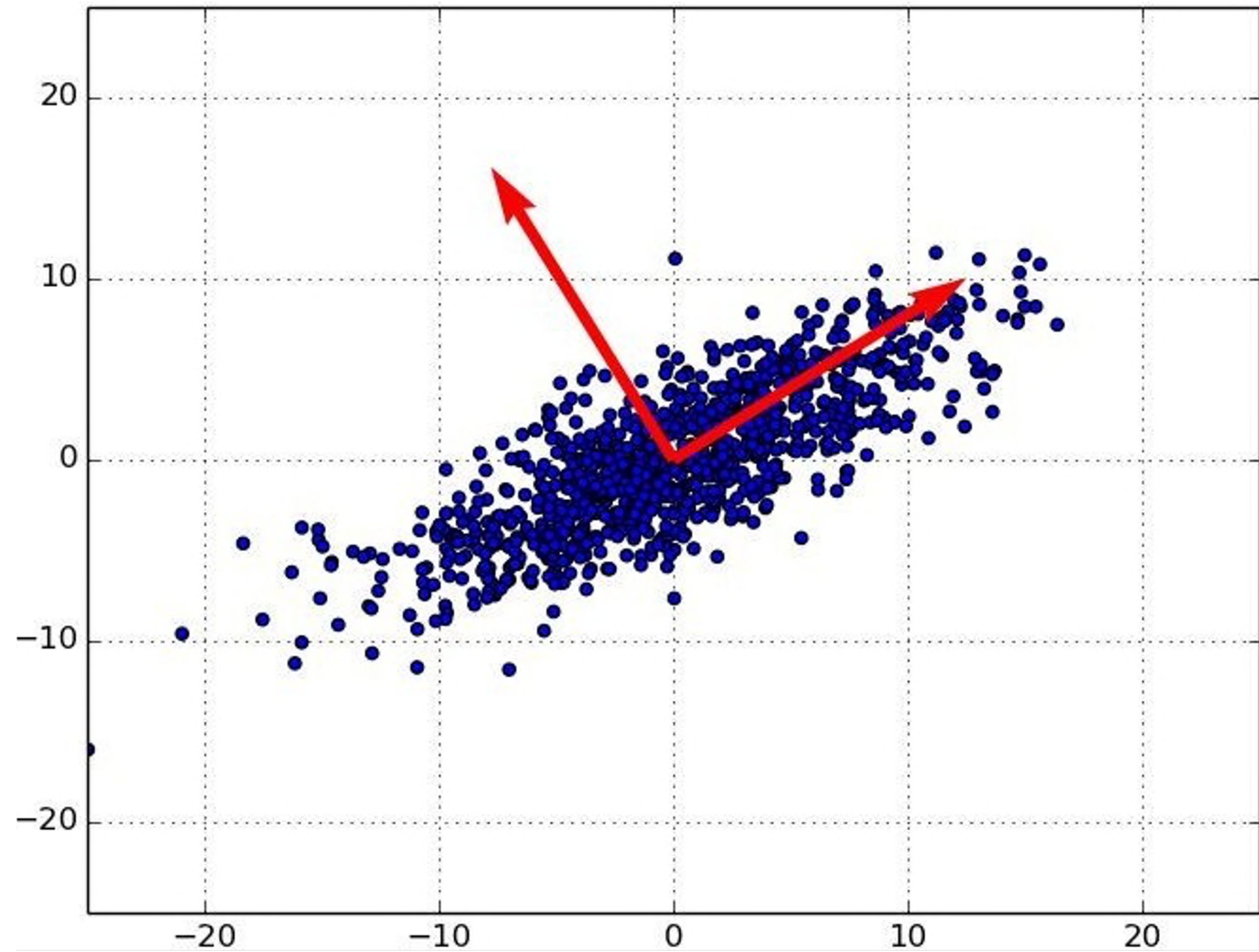


PCA – principal components analysis

Реконструированные изображения:



РСА с помощью scikit-learn



РСА – матрица ковариации

$$\text{Cov}(A) = \begin{bmatrix} \frac{\sum (x_i - \bar{X})(x_i - \bar{X})}{N} & \frac{\sum (x_i - \bar{X})(y_i - \bar{Y})}{N} \\ \frac{\sum (x_i - \bar{X})(y_i - \bar{Y})}{N} & \frac{\sum (y_i - \bar{Y})(y_i - \bar{Y})}{N} \end{bmatrix} = \begin{bmatrix} \text{Cov}(X, X) & \text{Cov}(Y, X) \\ \text{Cov}(X, Y) & \text{Cov}(X, Y) \end{bmatrix}$$

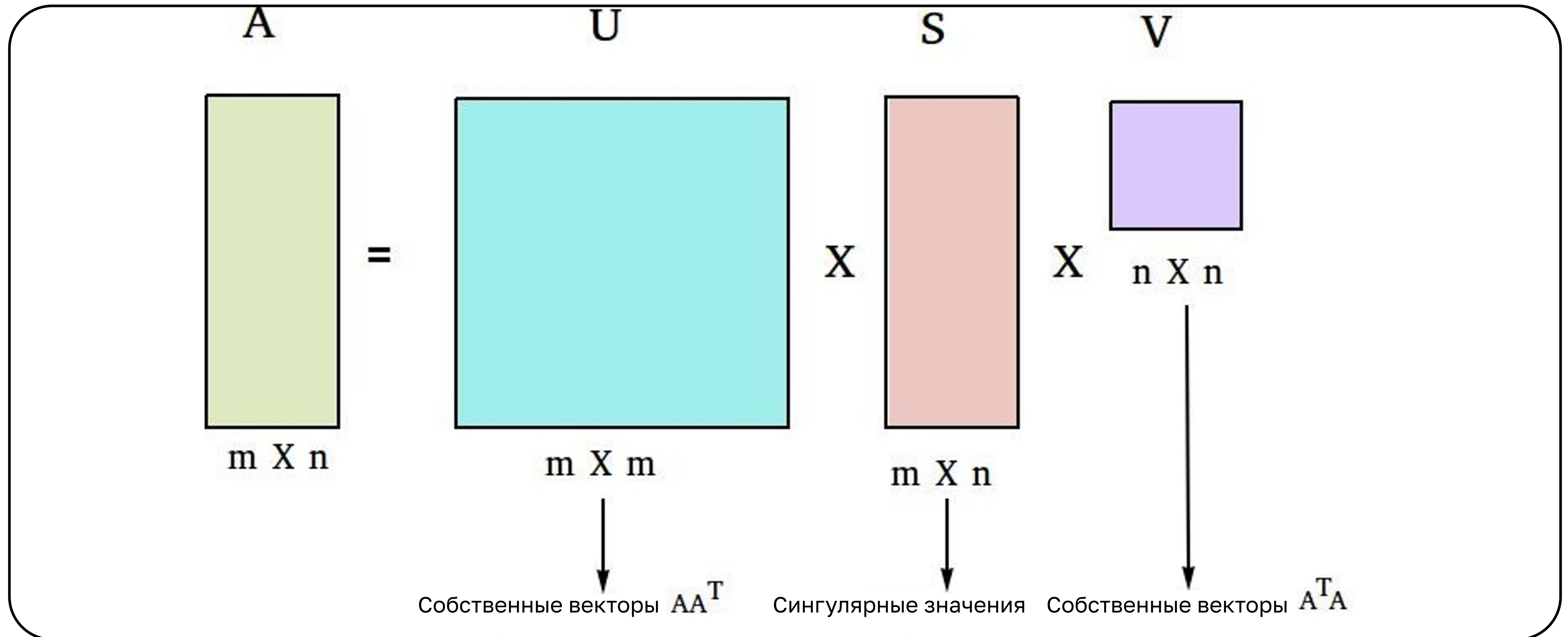
$\bar{X}, \bar{Y}$ – средние значения признаков по датасету

x_i, y_i – значения признаков у отдельного объекта

N – размер датасета

Выражение в верхних матрицах – ковариация выбранного признака, поэтому вычисления упрощаются до значений в нижней матрице

PCA – разложение SVD (singular value decomposition)



РСА – последовательность вычислений

- Подготавливаем данные, представляем изображения в виде векторов длиной $H \times W$
- Вычитаем среднее значение из каждой компоненты вектора
- Получаем собственные векторы в результате SVD разложения ковариационной матрицы изображений
- Выбираем размерность (число собственных векторов) на основе собственных значений

Выделение признаков изображений. Гистограммы цветов



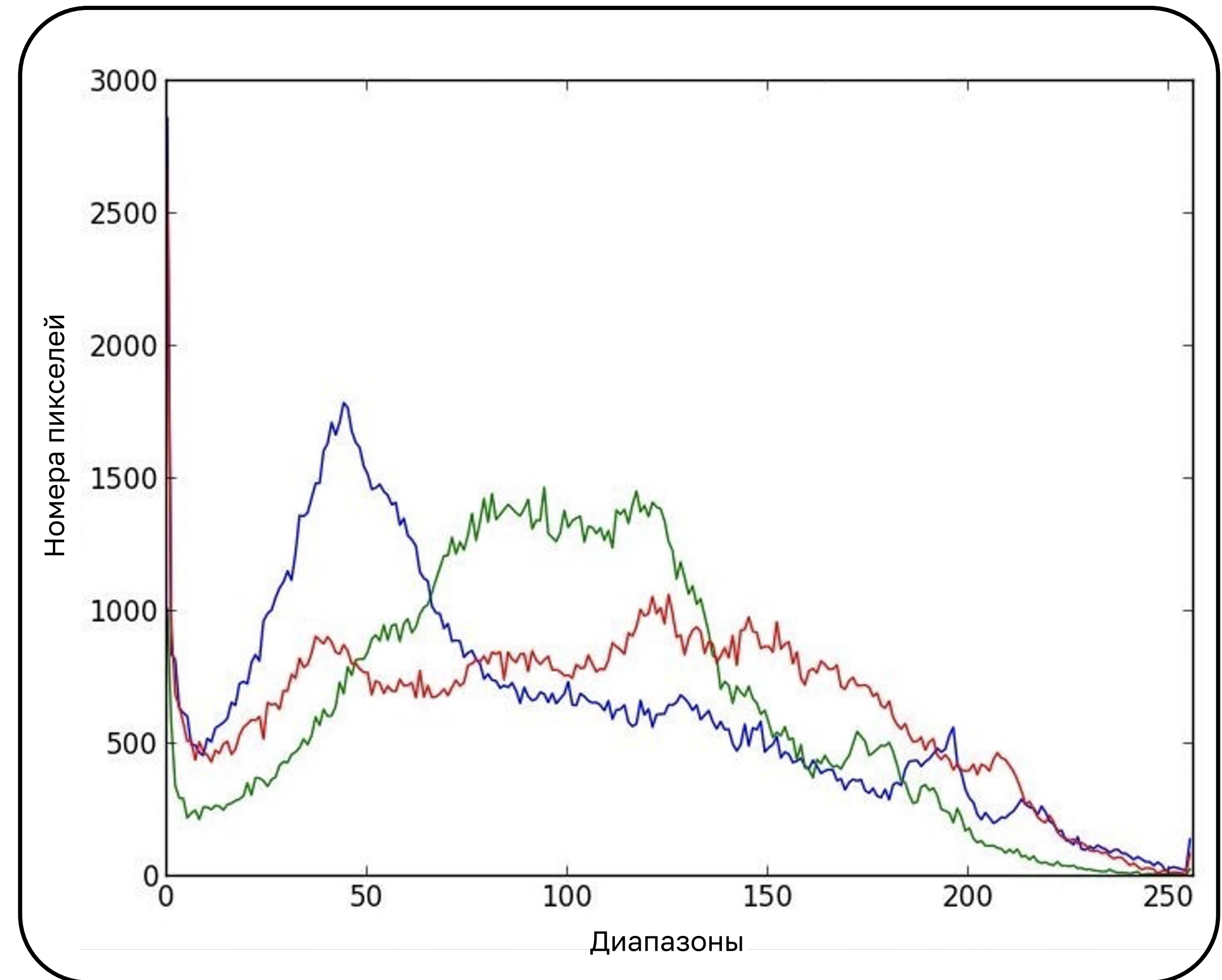
3

Гистограммы признаков изображения

- Представляют собой обобщённое описание изображения
- Как правило, гистограммы инвариантны к масштабу и повороту изображений
- Позволяют сравнивать изображения и находить похожие

Гистограммы цветов

Гистограмма для цветного изображения



Гистограммы цветов

- Разбиваем диапазон значений цвета (0...255) на фиксированное число ячеек (bins), например, с шагом 1
- Задаём срезы (каналы), в которых необходимо построить гистограмму
- Для каждой ячейки считаем число соответствующих пикселей на изображении
- Можно вычислять и в пространстве RGB, и в других цветовых пространствах, например, HSV

Гистограммы цветов

cv2.calcHist (images, channels, mask, histSize, ranges) → hist

- **images** — набор входных изображений для оценки гистограммы
- **channels** — каналы, по которым оцениваются гистограммы
- **mask** — маска, которая ограничивает область оценки гистограммы
- **histSize** — массив размеров гистограмм по каждому измерению
- **ranges** — диапазоны значений каждого измерения

Гистограммы цветов

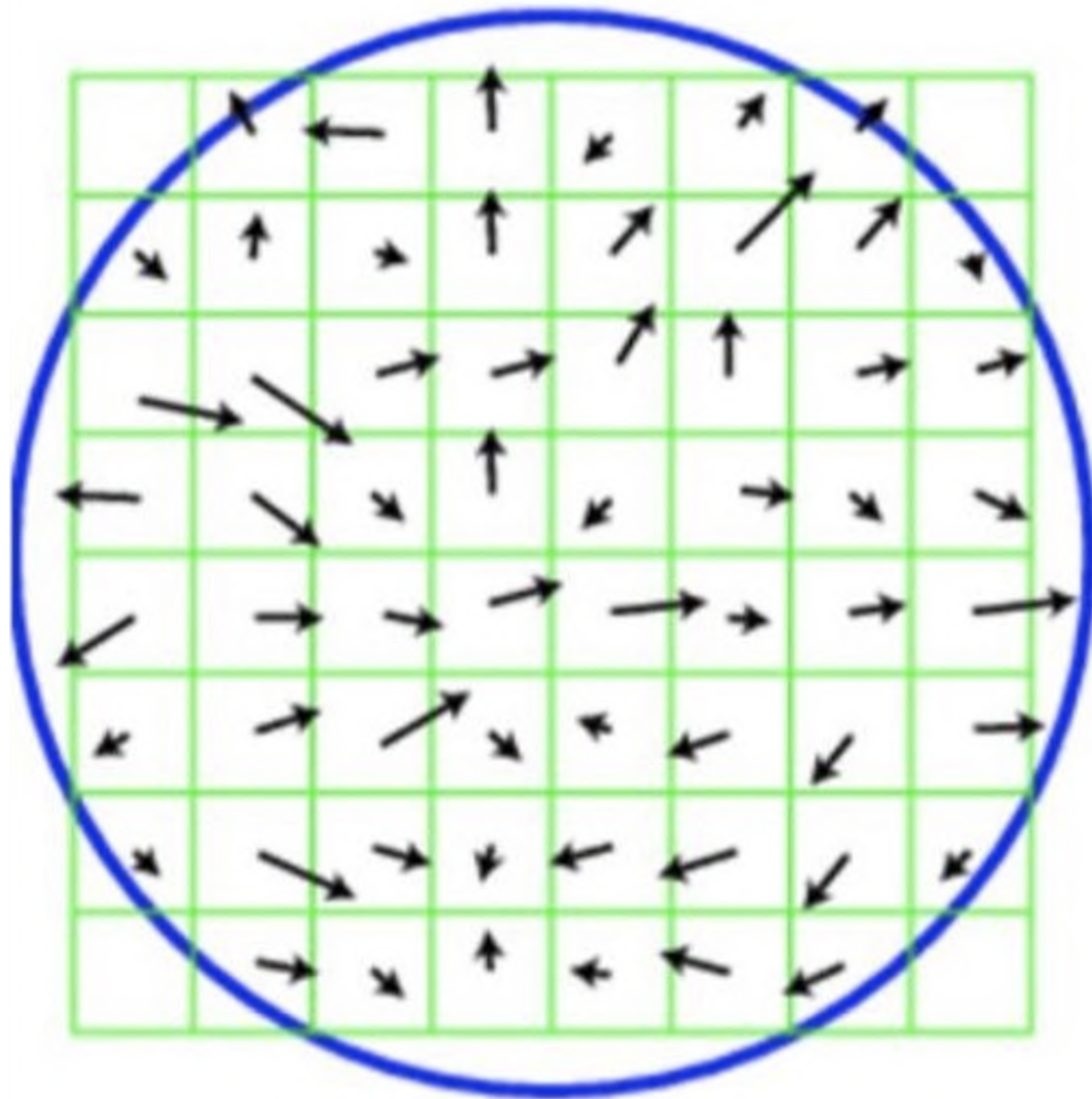
- Не зависят от изменения масштаба изображения
- Устойчивы к повороту и перспективным искажениям
- В цветовых пространствах HSV и HSL менее чувствительны к изменению яркости

Выделение признаков из изображений. Гистограммы градиентов

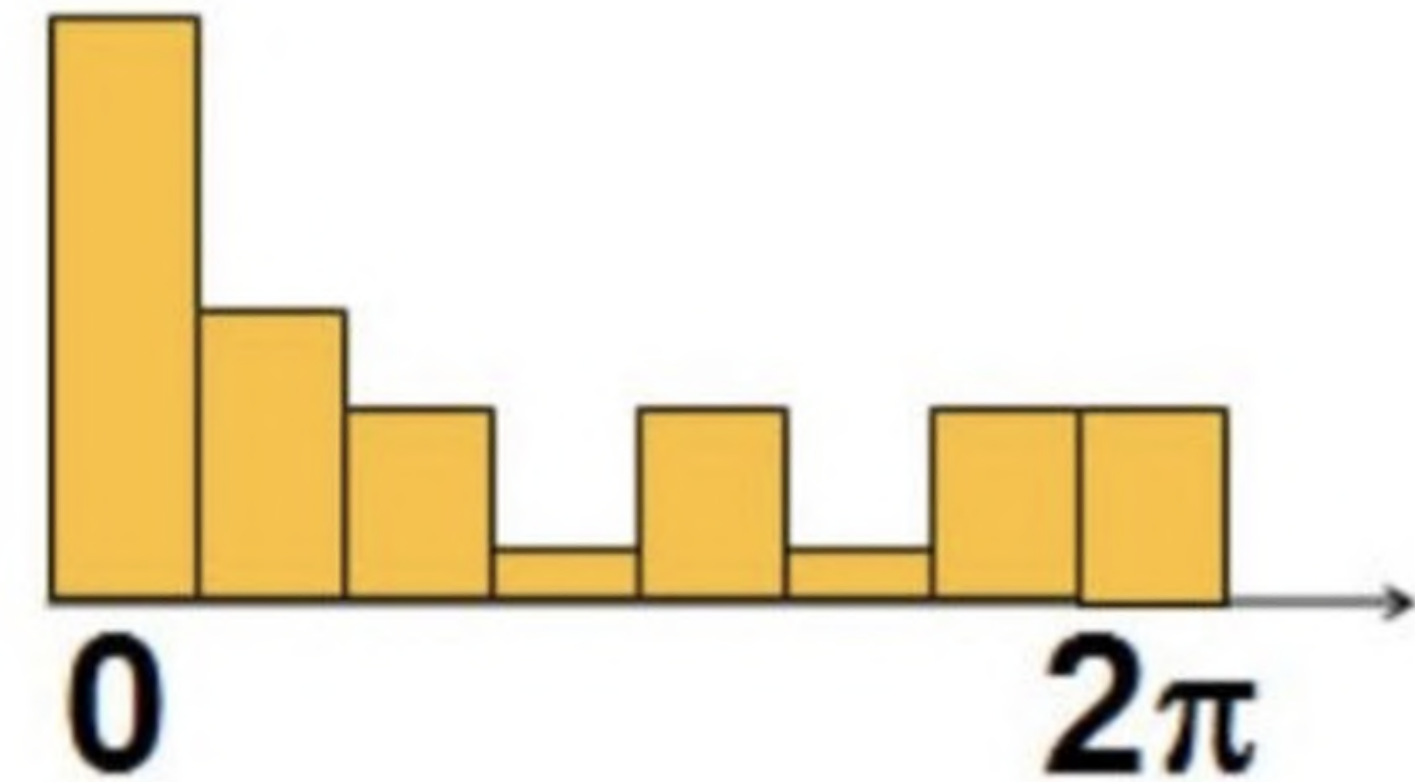


4

Гистограммы градиентов (HOG)



Градиенты изображения



Гистограмма угловых
распределений градиентов



Гистограммы градиентов (HOG)

- В каждой точке оцениваем составляющие градиента по осям x и y
- Определяем направление и длину вектора градиента
- Оцениваем гистограмму градиентов
- Полученные гистограммы нормализуют таким образом, чтобы вектор признаков был единичной длины

Оператор Собеля

Матрица свёртки оператора Собеля

-1	0	+1
-2	0	+2
-1	0	+1

X-фильтр

+1	+2	+1
0	0	0
-1	-2	-1

Y-фильтр

Оператор Собеля

Оригинал



Собель X



Собель Y



Гистограммы градиентов (HOG)

$$g = \sqrt{g_x^2 + g_y^2}$$

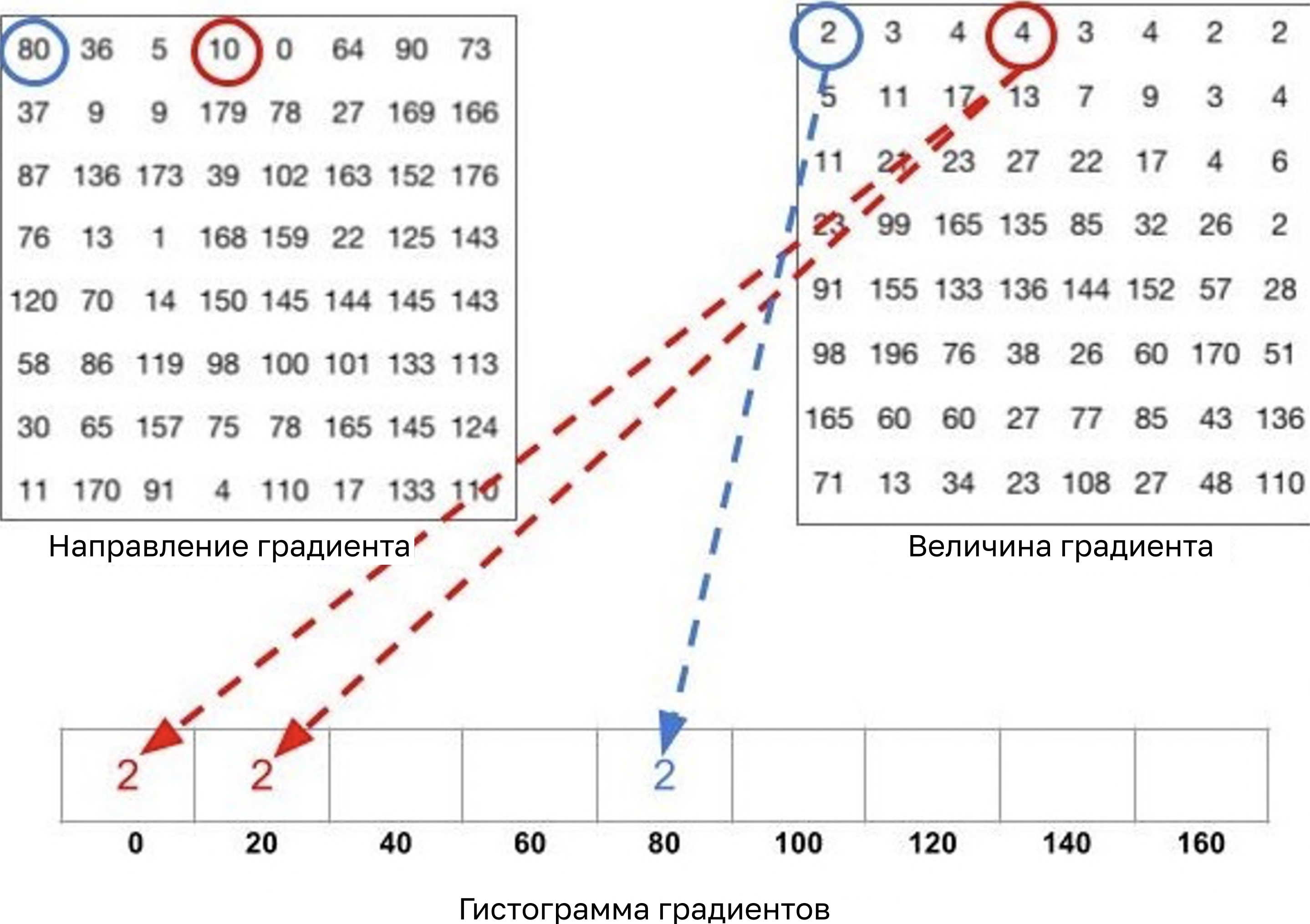
$$\theta = \arctan \frac{g_y}{g_x}$$

- **g**, **g_x**, **g_y** — длина вектора градиента и его составляющих
- **θ** — угол наклона градиента в полярной системе координат

Гистограммы градиентов (HOG)

- Как правило, гистограмму градиентов строят для диапазона углов 0...180
- При оценке гистограммы градиентов учитываются угол и длина вектора
- Чем больше длина вектора, тем больший вклад вносится в соответствующую ячейку гистограммы

Гистограммы градиентов (HOG)

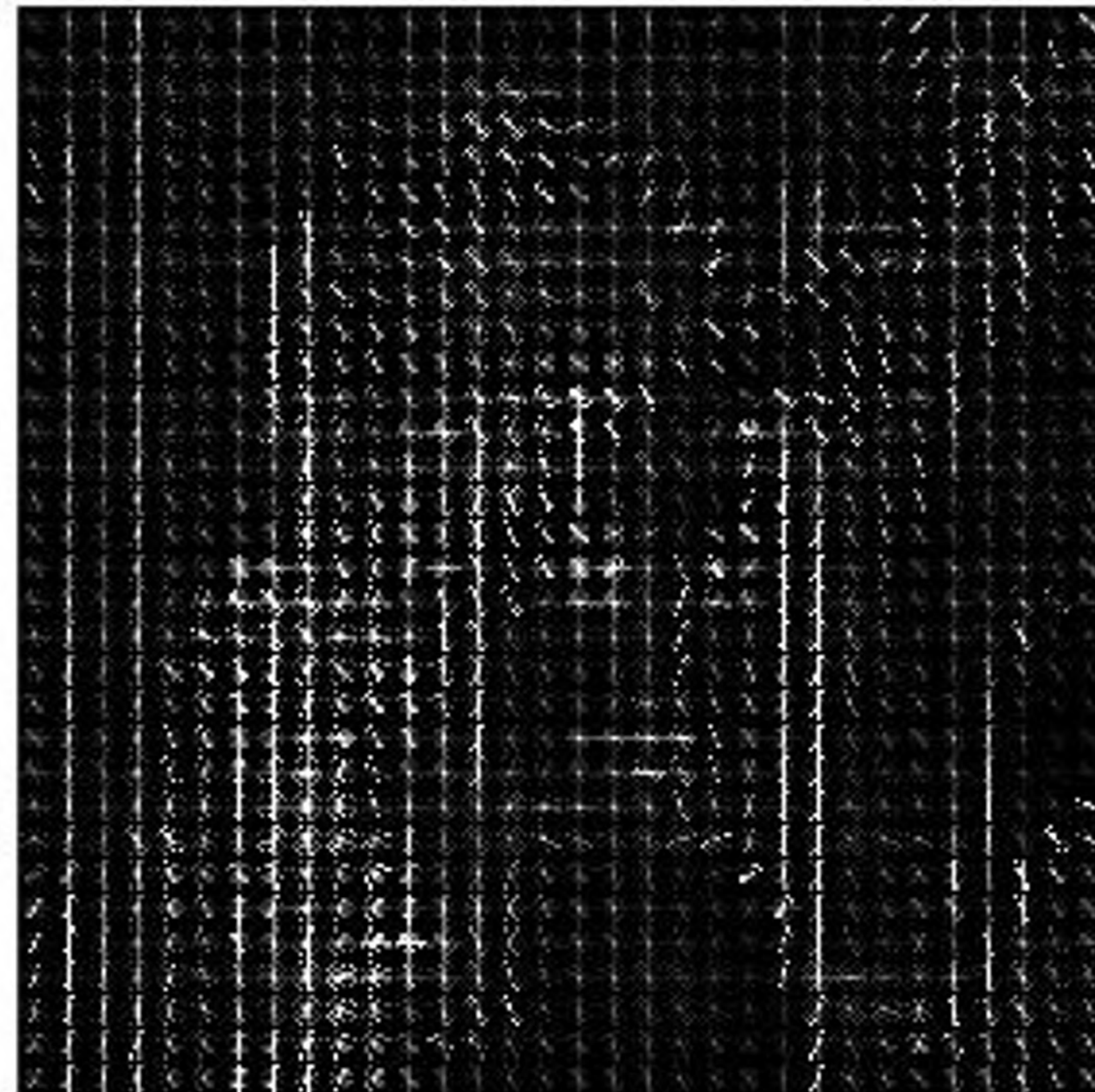


Гистограммы градиентов (HOG)

Входное изображение



Гистограмма ориентированных градиентов



Изображение и соответствующая гистограмма градиентов

Гистограммы градиентов (HOG)

`cv2.Sobel`(src, ddepth, dx, dy[, dst[, ksize]]) → dst

- **src** — входное изображение
- **ddepth** — тип данных для вычисления производной, например, `cv2.CV_64F`
- **dx/dy** — порядок производной по осям, как правило, 0 или 1
- **dst** — выходное изображение
- **ksize** — размер ядра фильтра 1, 3, 5, или 7

`cv2.cartToPolar`(x, y) → magnitude, angle

- **x,y** — векторы с координатами x и y
- **magnitude** — длины векторов
- **angle** — соответствующие углы

Гистограммы градиентов (HOG)

- Не чувствительны к изменению цвета
- Устойчивы к изменению яркости
- Устойчивы к изменению масштаба

Характерные точки



5

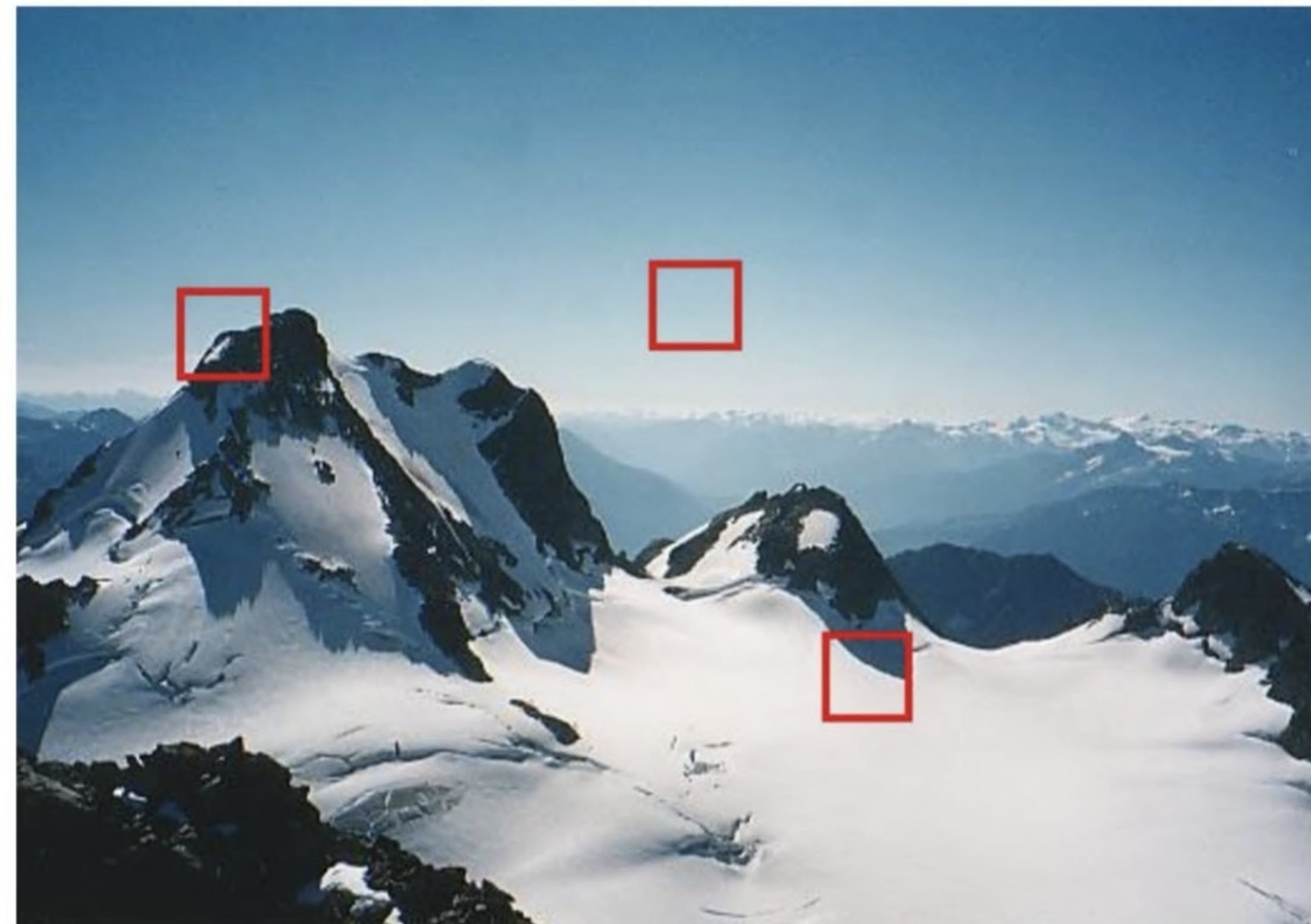
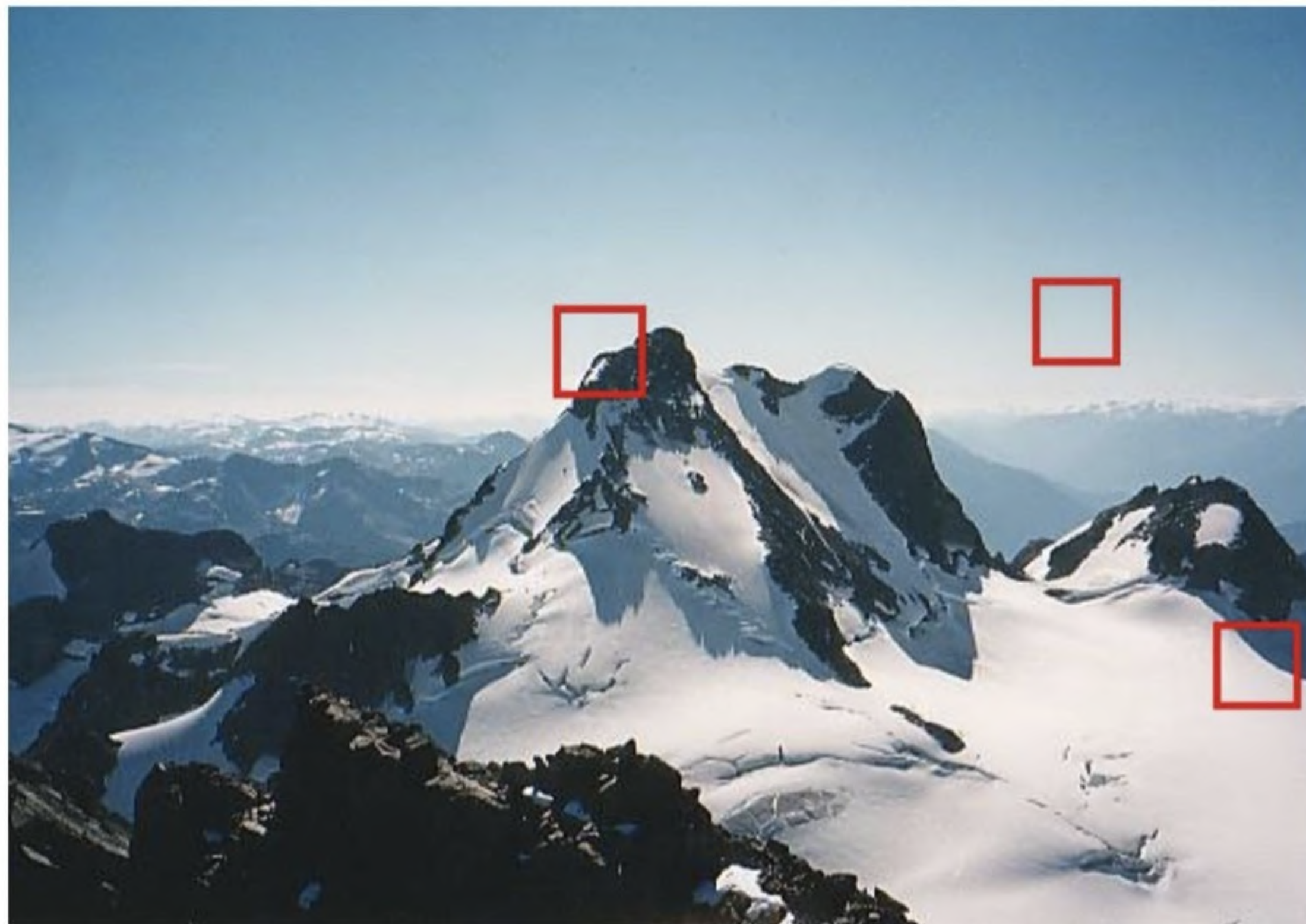
Характерные точки

- Позволяют находить одинаковые области (предметы) на разных изображениях
- Используются для склейки панорам и составления карт по спутниковым снимкам

Характерные точки

- Точка, обладающая уникальными свойствами
- Положение точки на изображении однозначно определяется по её свойствам (дескриптору)
- Дескриптор точки вычисляется на основе её окружения
- Дескриптор характерной точки инвариантен к изменениям изображения (освещённость, поворот, масштабирование)

Характерные точки



Этапы поиска и мэтчинга характерных точек

- 1 Определяем области на изображении, которые наиболее вероятно содержат характерную точку
- 2 Вычисляем дескрипторы точки по каждой из областей
- 3 Находим точки с одинаковыми дескрипторами для мэтчинга изображений

Поиск характерной точки на изображении

Как понять что выбранная область содержит характерную точку:

- область вокруг характерной точки должна сильно варьироваться
- в области характерной точки небольшой сдвиг изображения должен приводить к существенному различию с исходным изображением

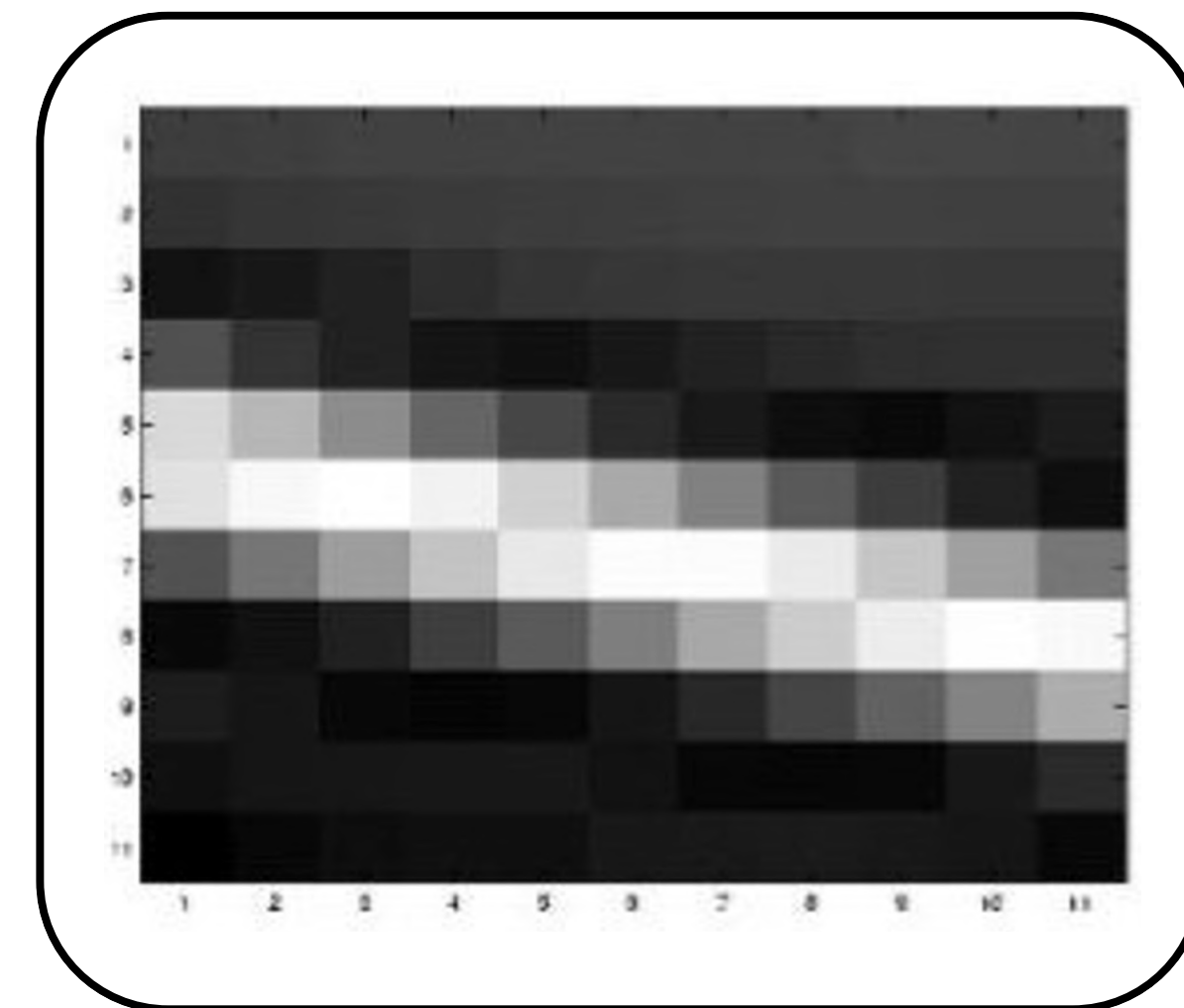
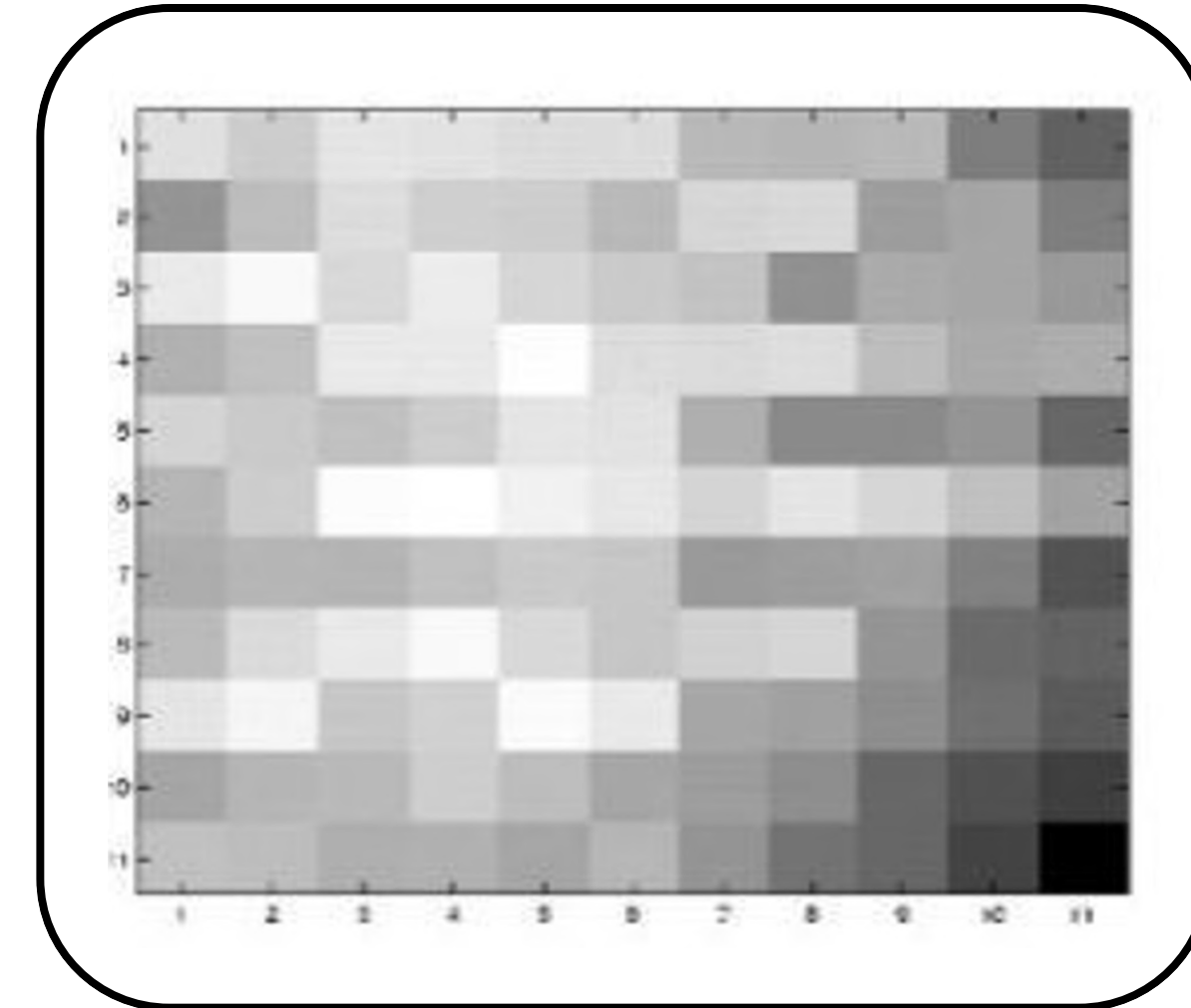
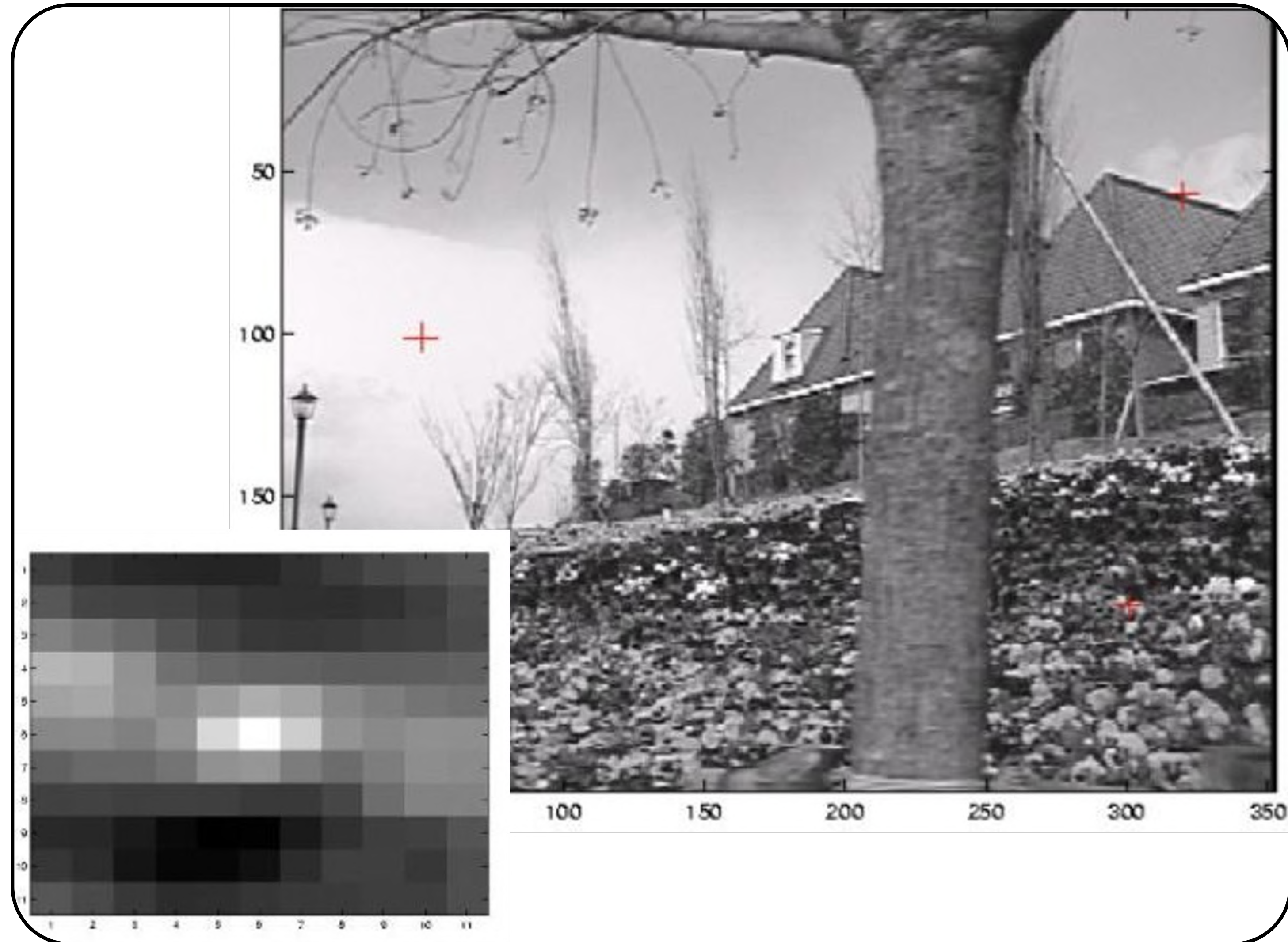
Автокорреляция

$$E_{AC}(\Delta \mathbf{u}) = \sum_i w(\mathbf{x}_i) [I_0(\mathbf{x}_i + \Delta \mathbf{u}) - I_0(\mathbf{x}_i)]^2$$

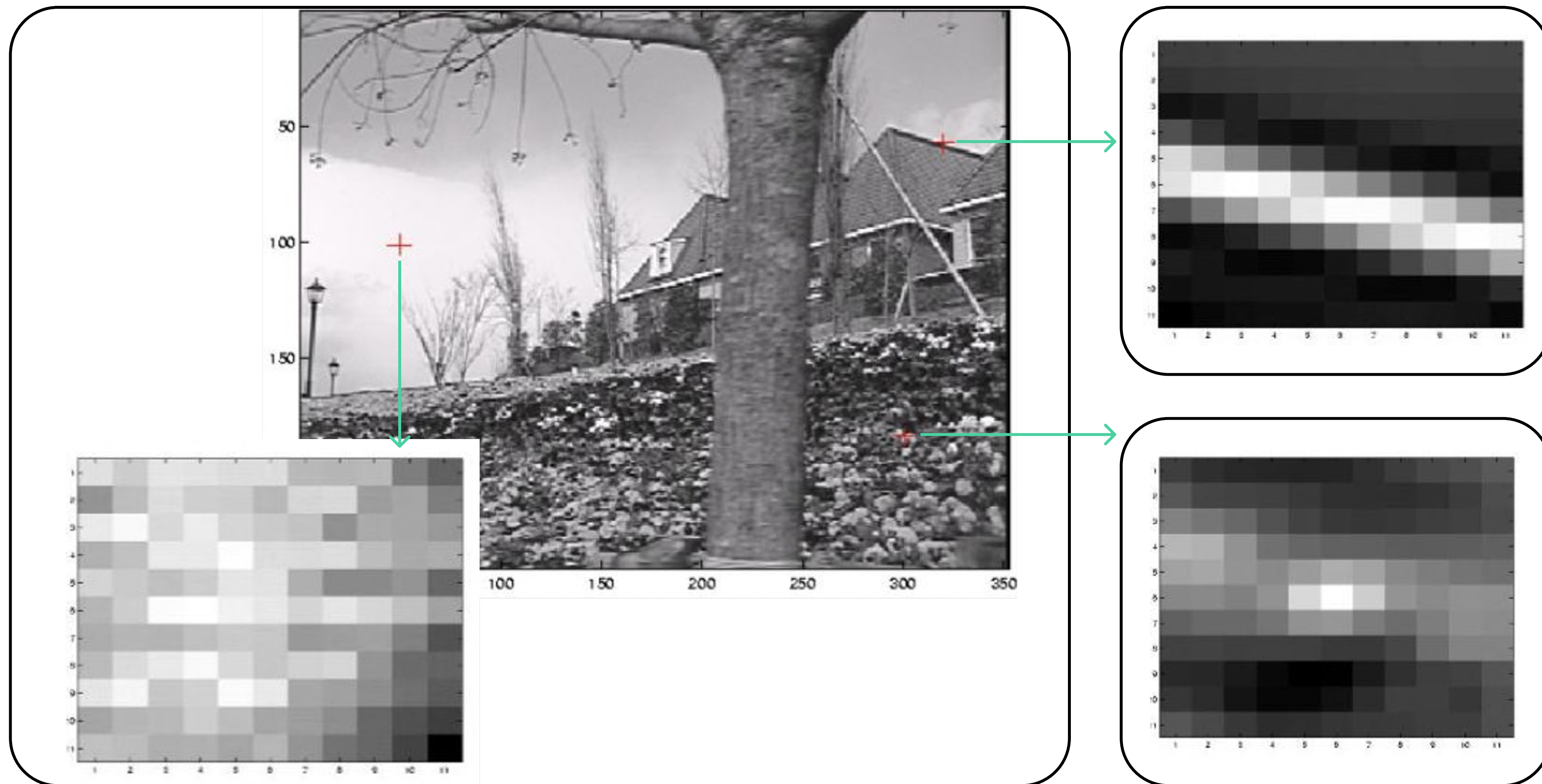
Вычисление автокорреляции

- $\mathbf{du}$ — вектор смещения по осям x и y
- $\mathbf{x}_i$ — вектор координат пикселя изображения
- $\mathbf{w}$ — окно или фильтр (гауссовский)
- I — исходное изображение

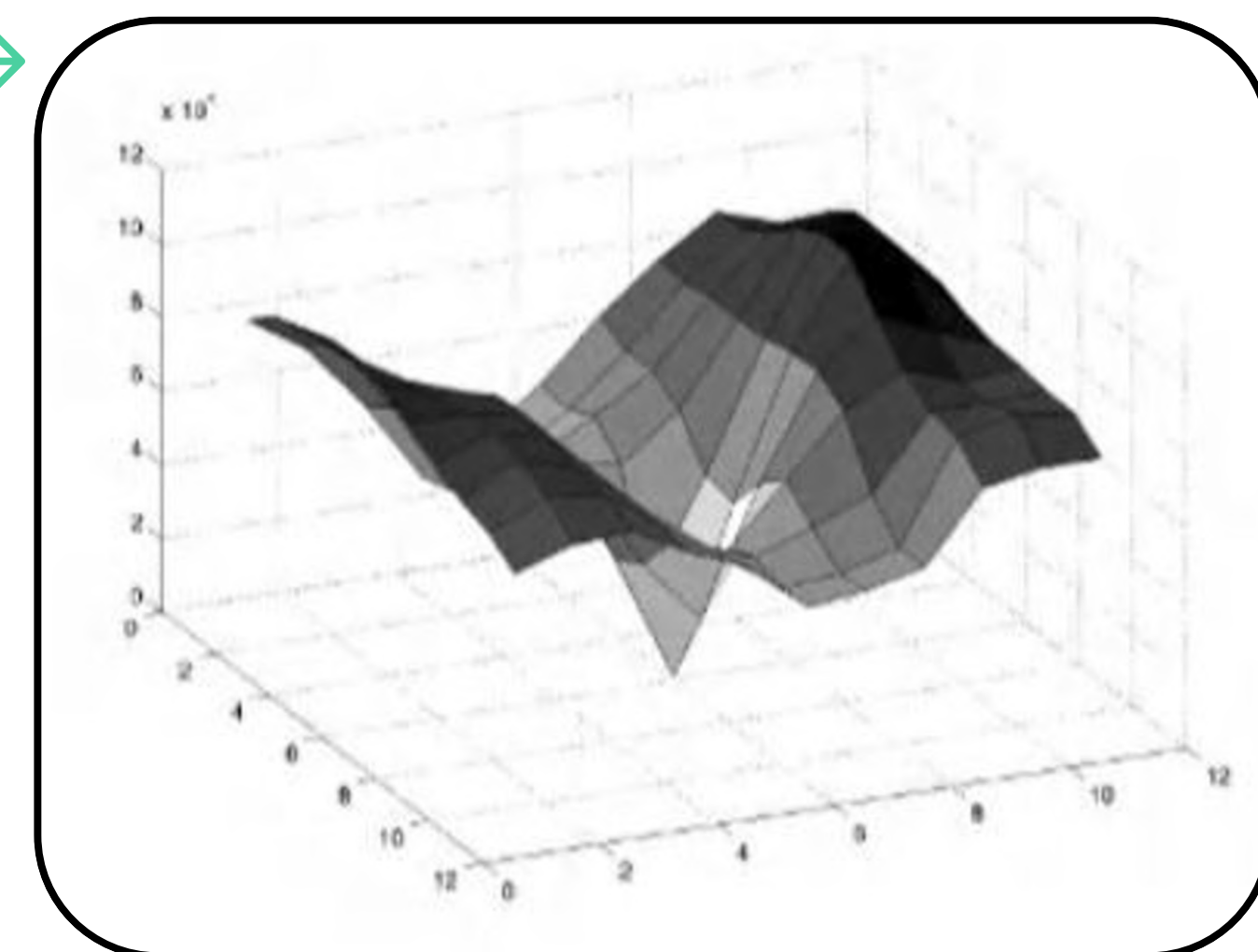
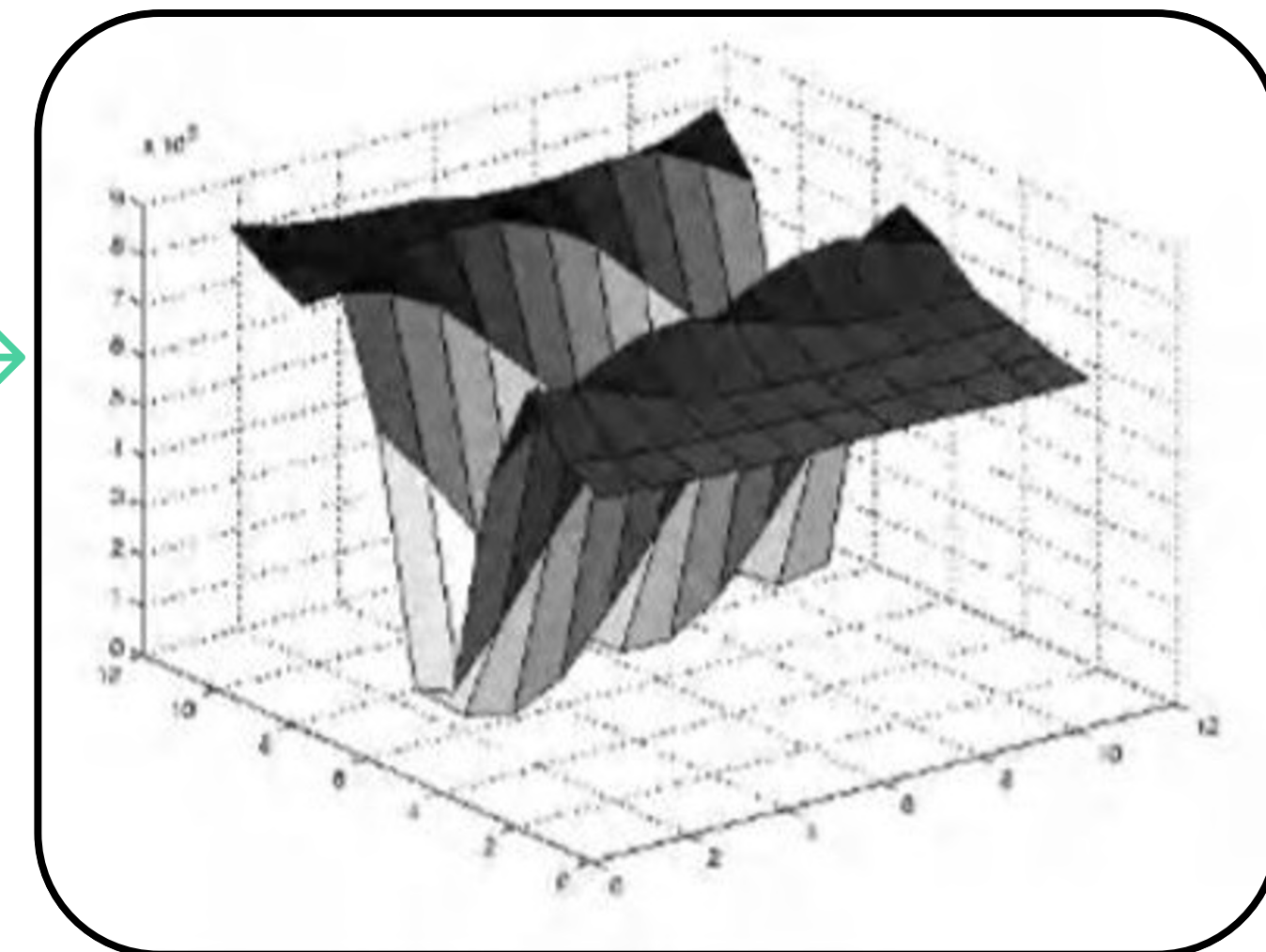
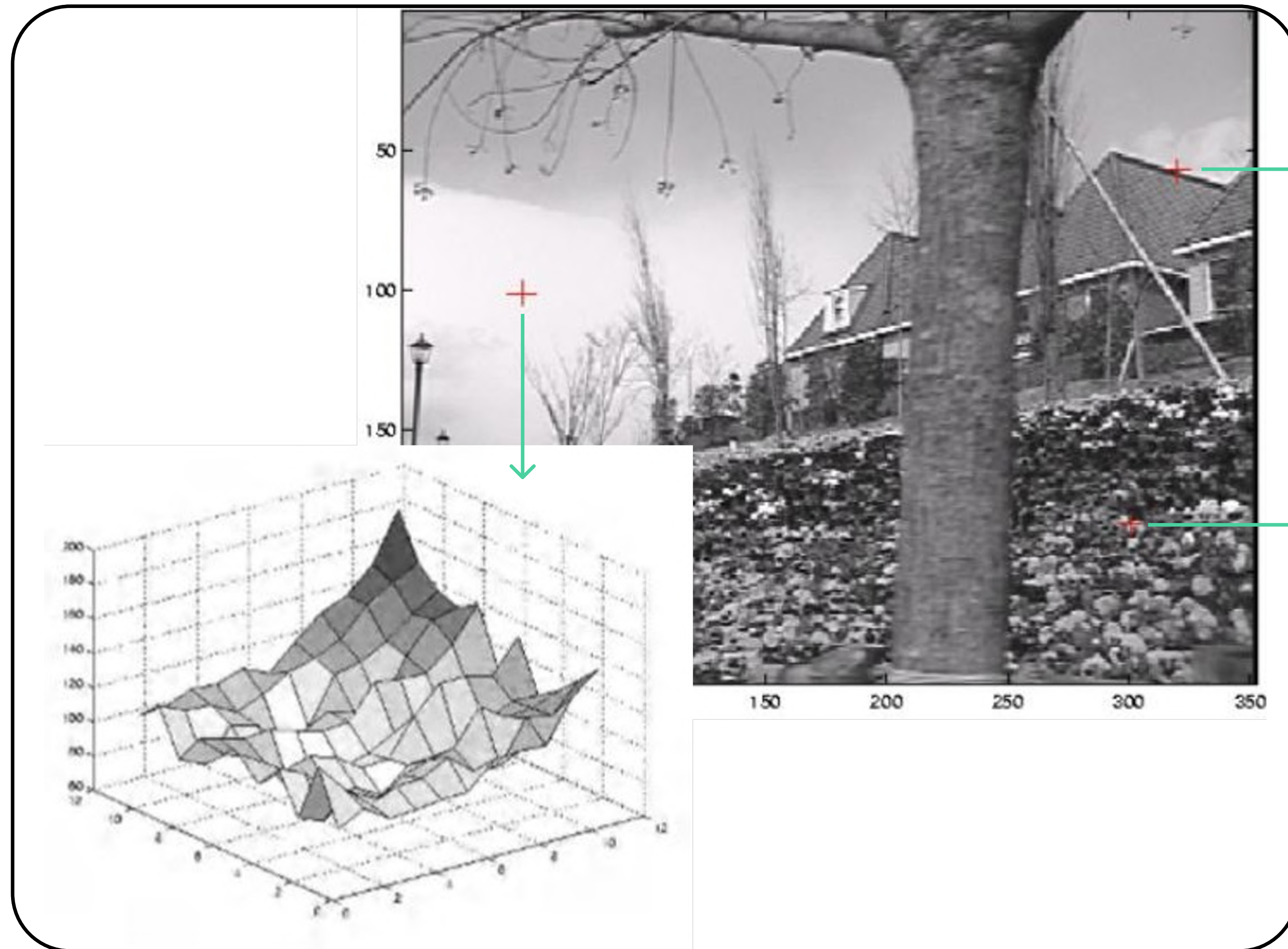
Матрица автокорреляции



Матрица автокорреляции



Матрица автокорреляции



Поиск характерной области на изображении

- Необходима количественная мера для поиска области с характерной точкой
- Большая часть мер основана на собственных числах матрицы автокорреляции в анализируемой области
- Т. к. область вокруг точки должна сильно варьироваться, то нас интересуют области с большими значениями собственных чисел матрицы автокорреляции

Поиск характерной точки на изображении

- ① Строим автокорреляционную матрицу изображения
- ② Для каждой точки изображения вычисляем собственные числа в соответствующей окрестности матрицы автокорреляции
- ③ Оставляем точки с локальным максимумом меры (Non-Maximum Suppression)
- ④ Полученные области содержат характерные точки

SIFT – scale invariant feature transform

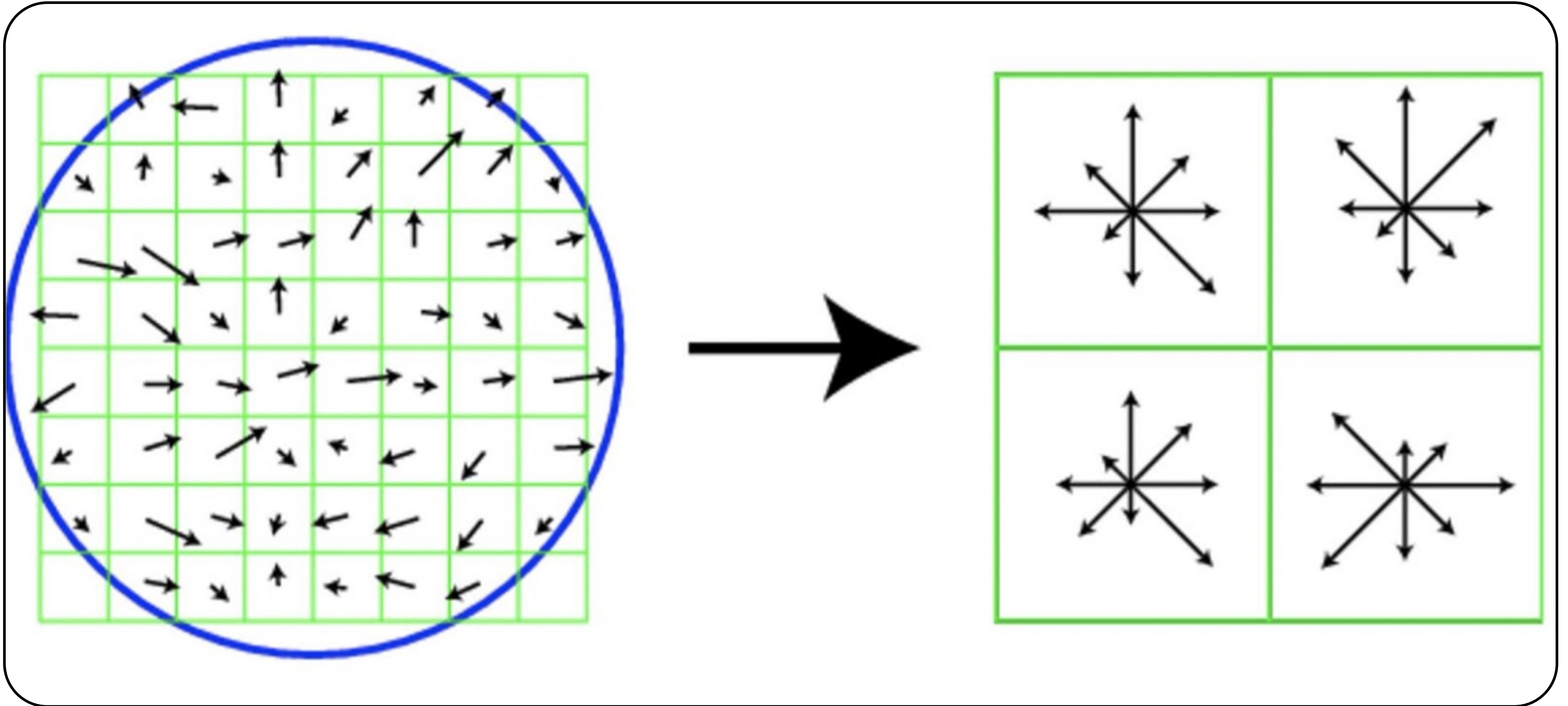
SIFT (scale invariant feature transform) – масштабно-инвариантная трансформация признаков.

- Дескриптор основан на построении гистограммы градиентов (HOG)
- В окрестности характерной точки выделяется область размером 16x16 пикселей
- Для каждого пикселя оценивается вектор градиента
- Длина вектора градиента взвешивается гауссовским фильтром таким образом, чтобы пиксели, удалённые от характерной точки, имели меньший вес

SIFT – scale invariant feature transform

- Исходная область 16x16 разбивается на части размера 4x4
- Для каждой части строится гистограмма градиентов с 8 ячейками
- В результате получается вектор из 128 признаков
- Полученный вектор нормируется до единичной длины

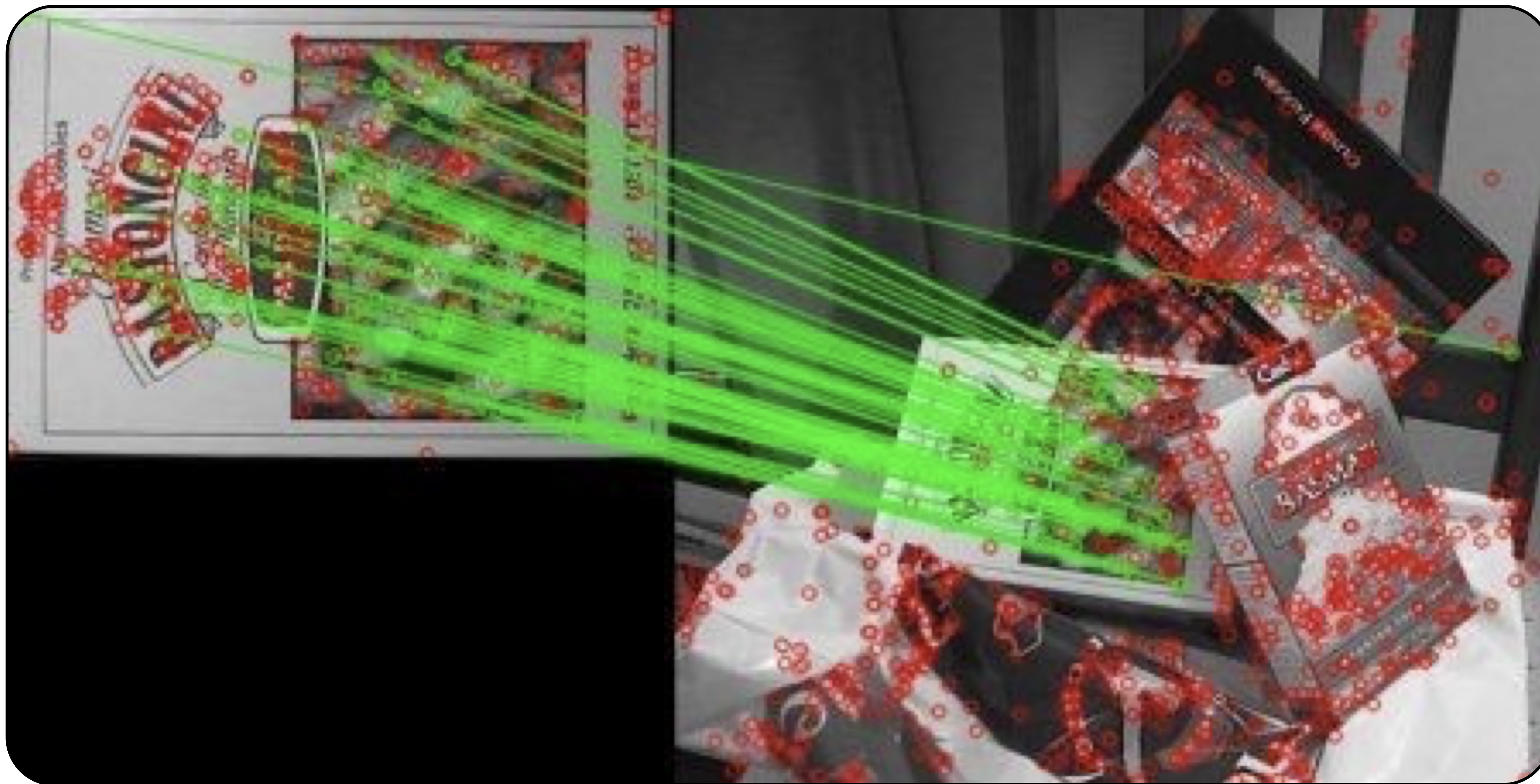
SIFT – scale invariant feature transform



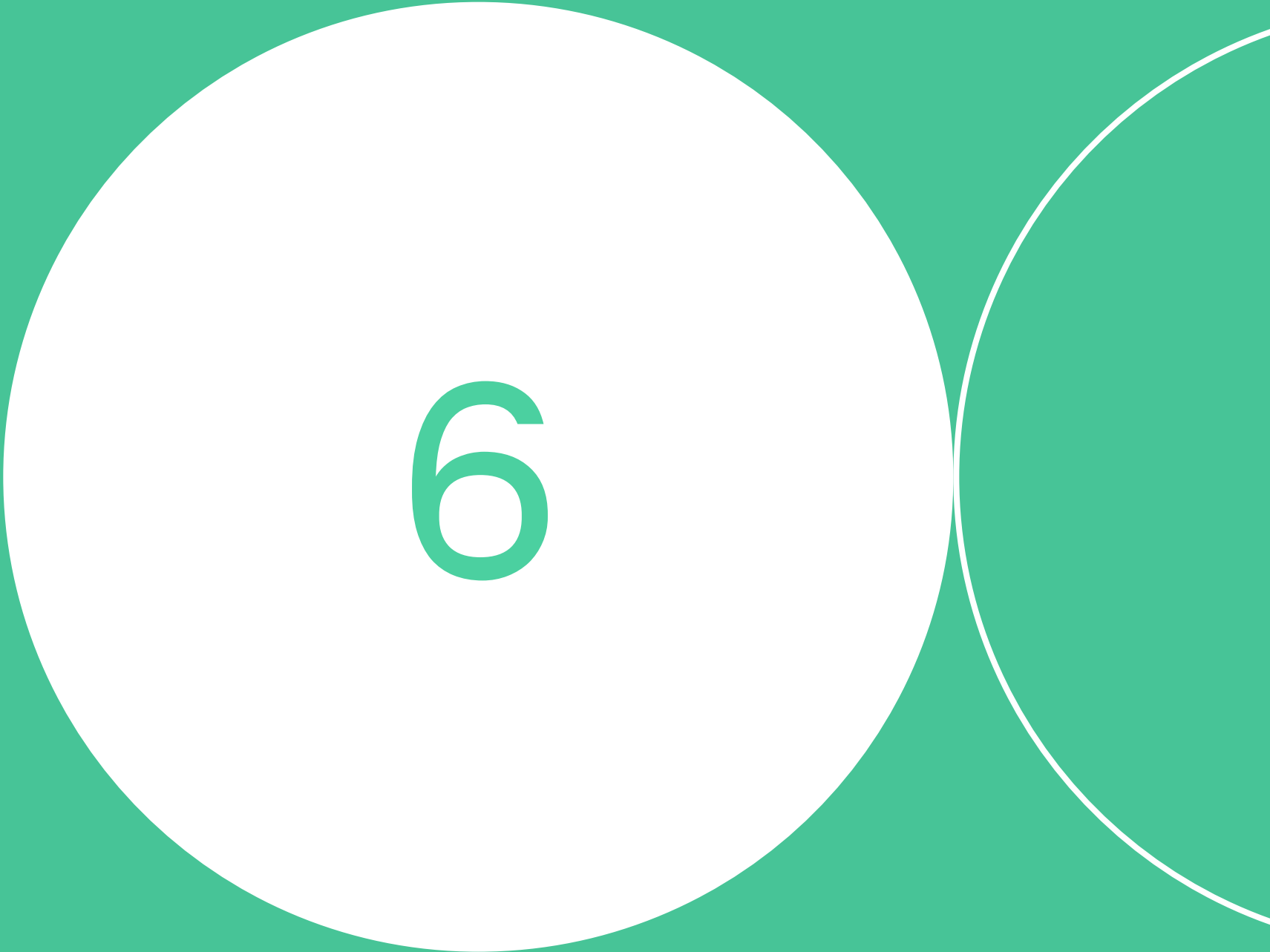
Мэтчинг характерных точек

- Выбрать меру расстояния для дескрипторов — евклидова мера (L2), L1, Hamming
- Попарное сравнение всех точек — полный перебор (долго)
- Индексация перед поиском и поиск по индексу:
 - поиск точек в окрестности — `kdtree`
 - хеширование точек таким образом, чтобы точки с похожими дескрипторами оказывались рядом — `locality sensitive hashing`

Мэтчинг характерных точек



Нейронные сети в компьютерном зрении



6

Нейронные сети

- Нейронные сети состоят из нейронов
- Каждый нейрон линейно разделяет пространство на два подпространства
- Выводы нейронов снабжаются нелинейными функциями (функции активации)

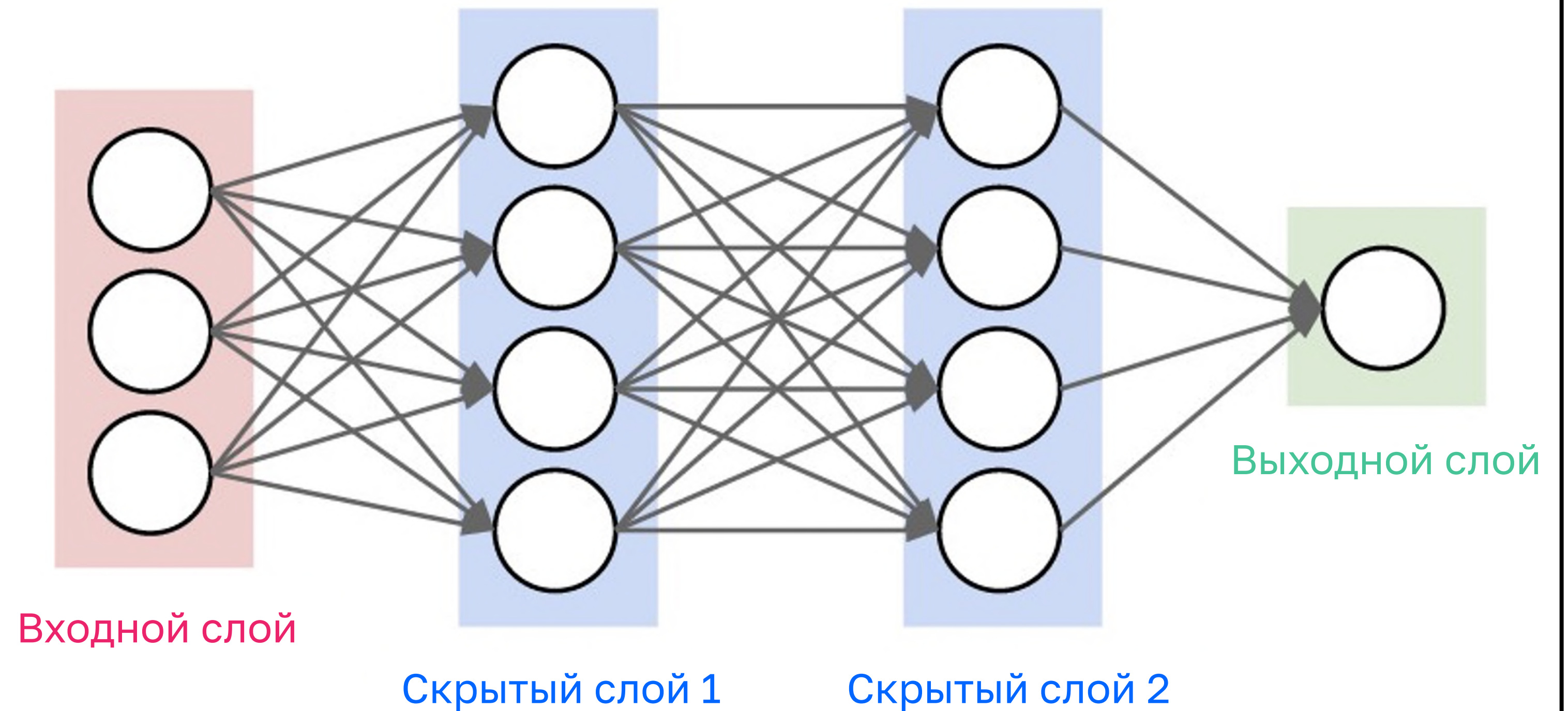
Нейронные сети

$$x = (x_1 \dots x_n), y_i = f\left(\sum_j W_{ij}x_j + b_i\right)$$

$$y^1 = f_1(W_1x + b_1)$$

$$y^2 = f_2(W_2y^1 + b_2)$$

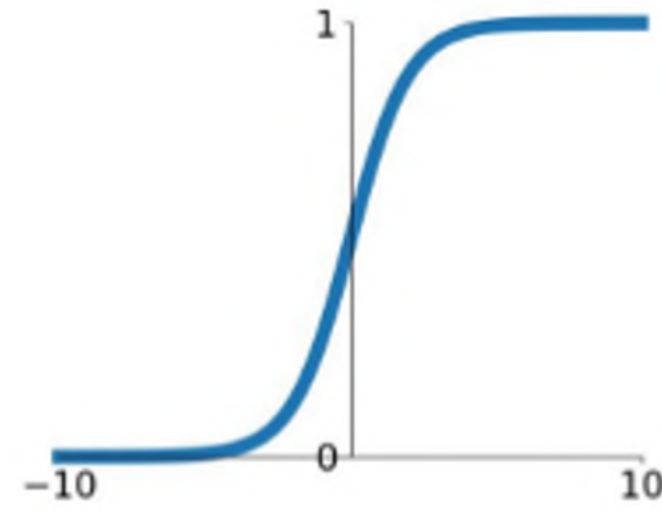
$$y^n = f_n(W_ny^{n-1} + b_n)$$



Нейронные сети. Функции активации

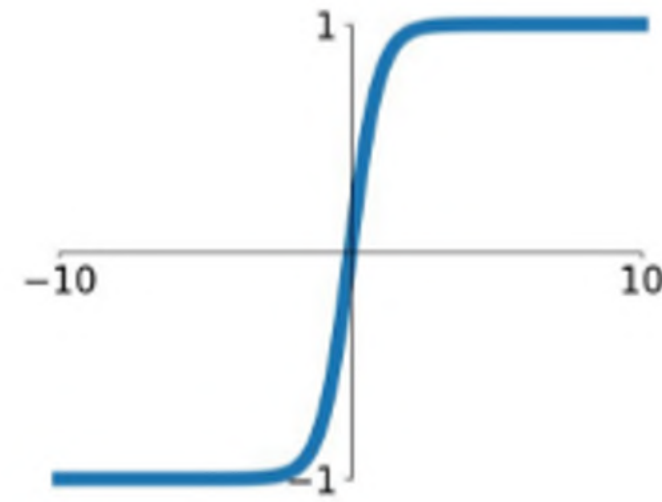
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



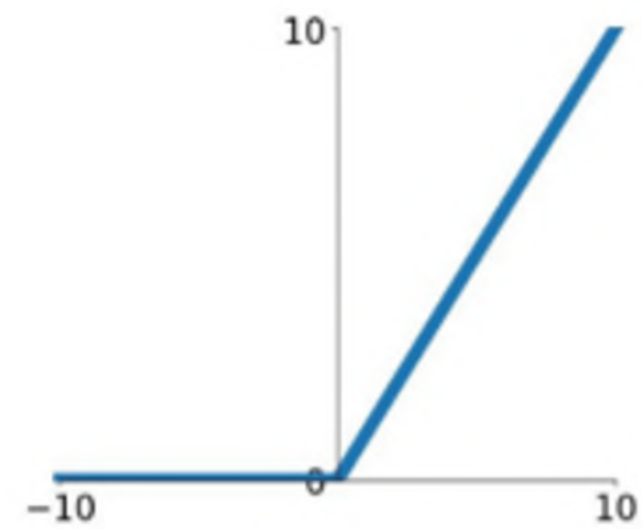
tanh

$$\tanh(x)$$



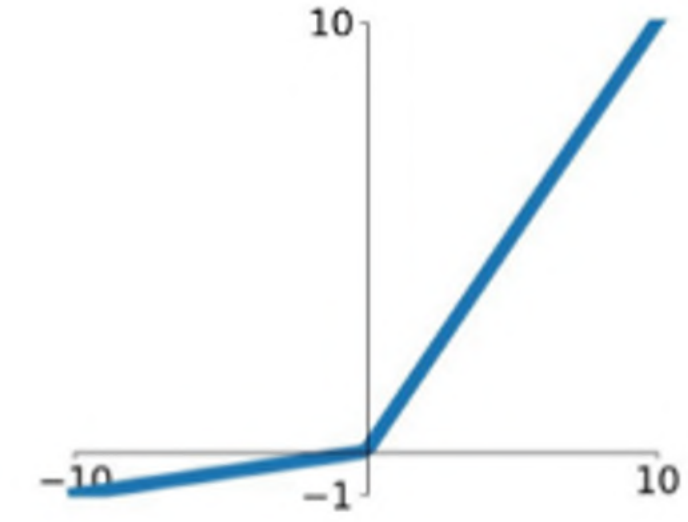
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

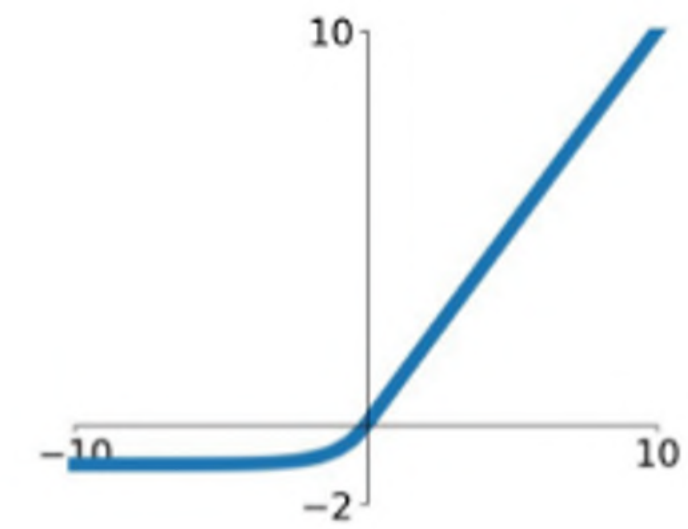


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

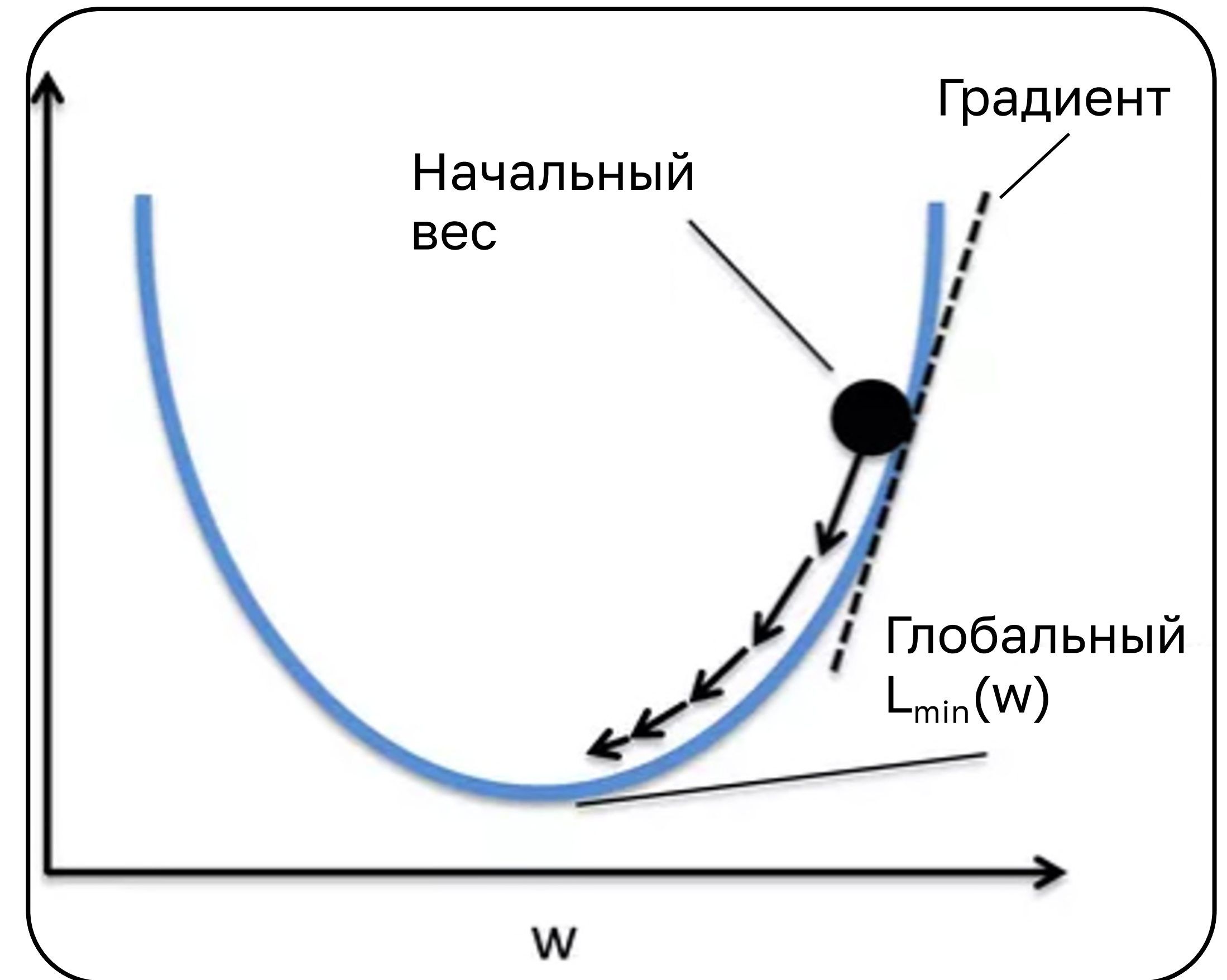
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

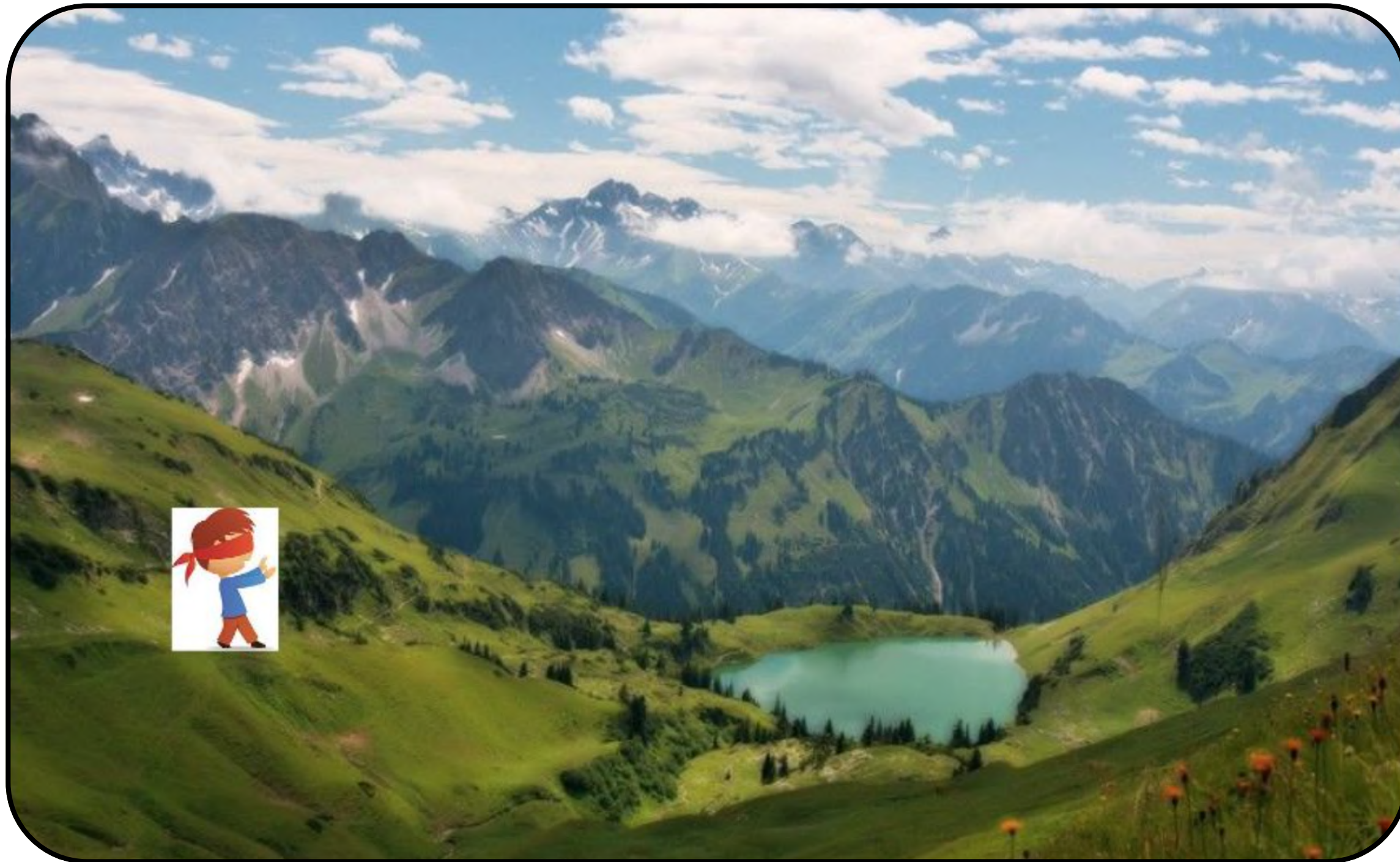


Нейронные сети. Обучение

- Минимизируем $L(x, y, w)$:
$$L(x, y, w) = (f(x, w) - y)^2 \rightarrow \min_w$$
- Инициализируем W_0
- На каждом шаге считаем градиент функции L : $L'_w(x, y, w_i)$
- Делаем шаг $w_{i+1} = w_i - \alpha L'_w(x, y, w_i)$



Нейронные сети. Обучение



Нейронные сети. Обратное распространение ошибки

- Давайте распишем многослойный персептрон $f(x, w)$:

$$f(x, w) = f_n(W_n f_{n-1}(W_{n-1} \dots f_1(W_1 x)))$$

- Чтобы сделать шаг, нам нужно знать все $\frac{\partial L}{\partial W_1} \dots \frac{\partial L}{\partial W_n}$

Нейронные сети. Обратное распространение ошибки

Функция потерь для обучения нейросети:

$$L(x, y, w) = (f(x, w) - y)^2$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial w}; \quad \frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}$$

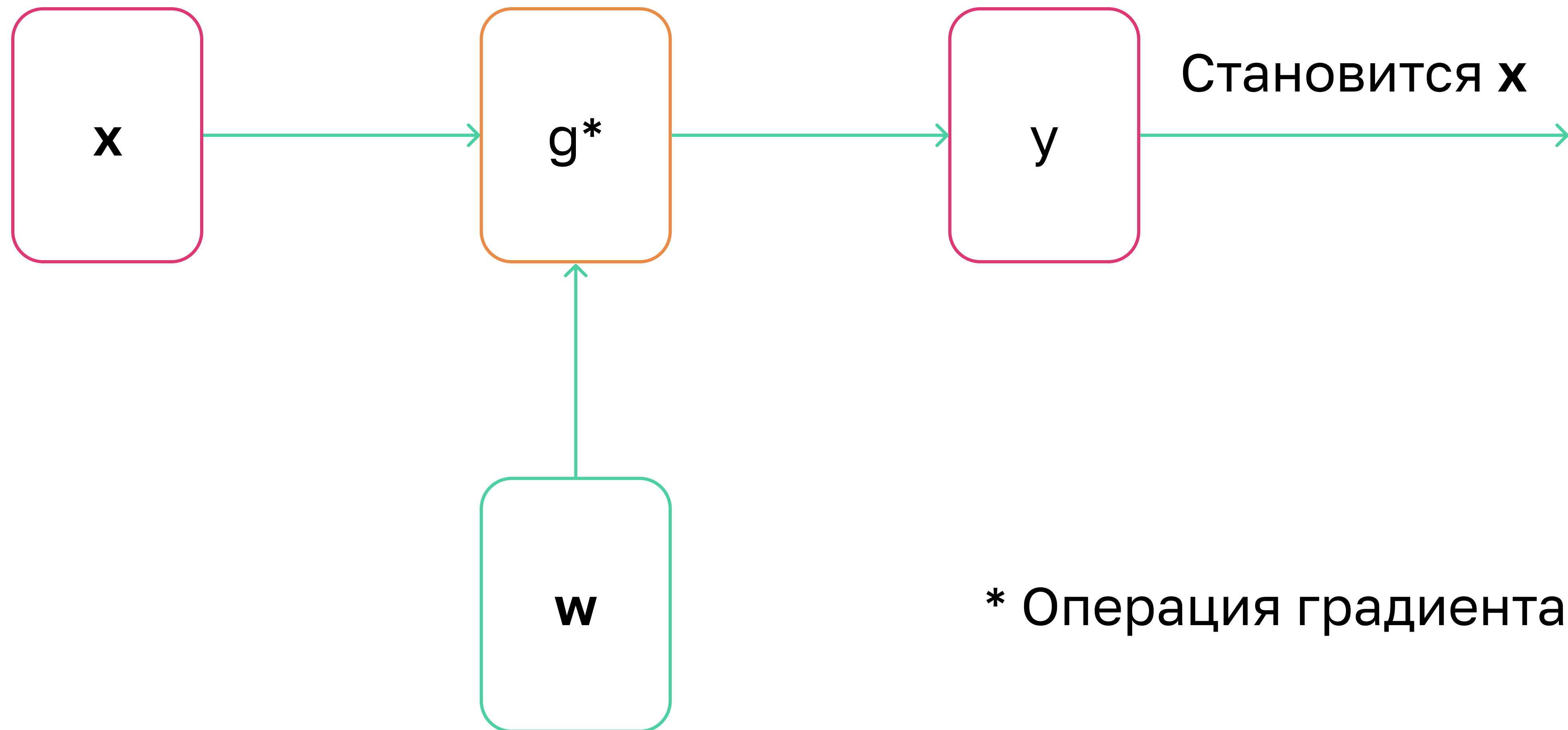
$$y = w \cdot x \Rightarrow \frac{\partial y}{\partial w} = x \Rightarrow \frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} x$$

$$y = w \cdot x \Rightarrow \frac{\partial y}{\partial x} = w \Rightarrow \frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} w$$

$\frac{\partial L}{\partial w}$ — градиент по параметрам нейросети (w).

L состоит из двух множителей: $\frac{\partial L}{\partial y}$ — градиент функции потерь по выходу нейросети,
 $d \frac{\partial y}{\partial w}$ — градиент выхода нейросети по параметрам

Нейронные сети. Обратное распространение ошибки



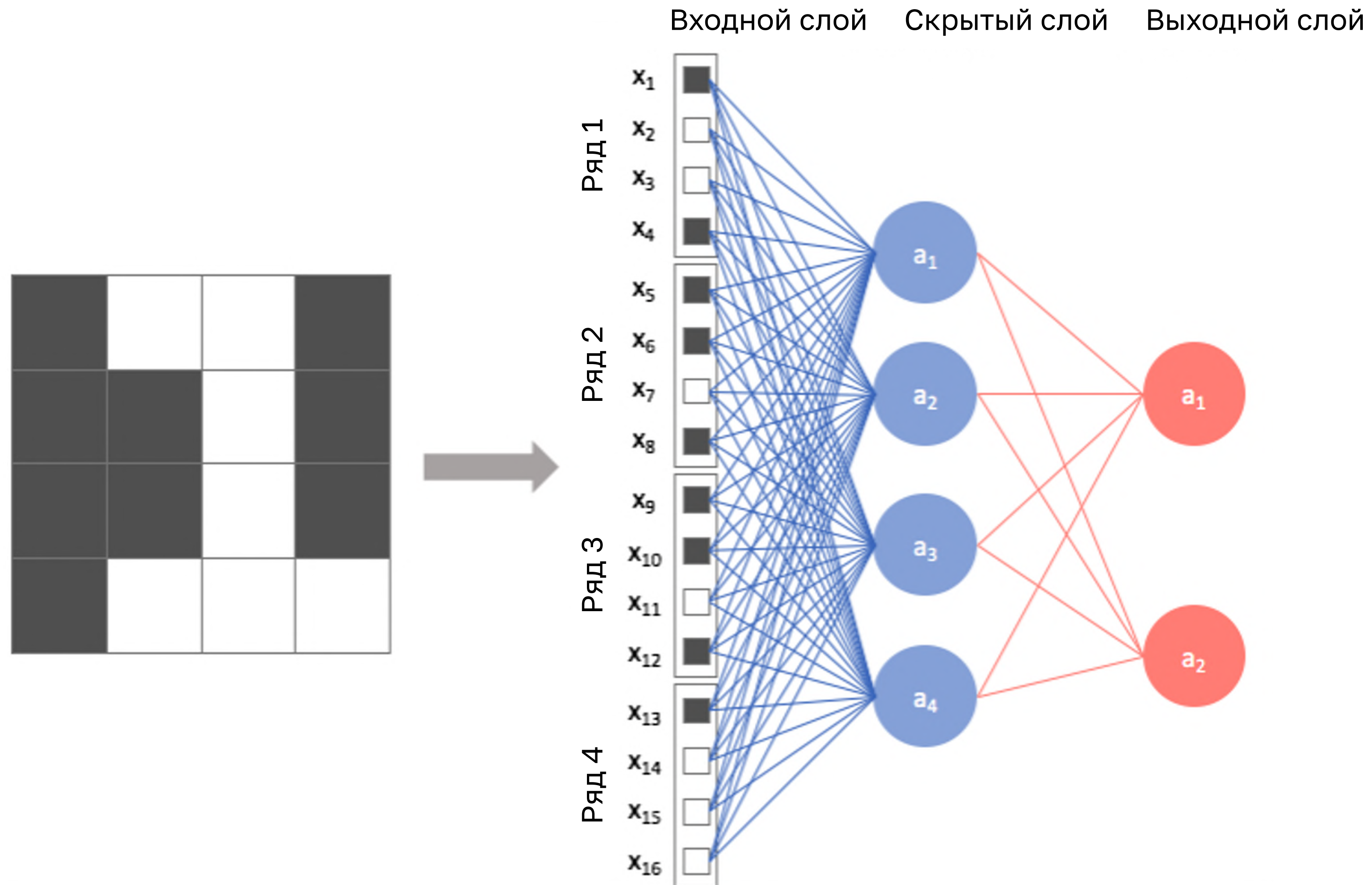
Нейронные сети. Обработка изображений

Чтобы обработать изображения нейронными сетями, требуется:

- преобразовать матрицу пикселей изображений в один длинный столбец
- подать на вход нейросети столбец, трактуя его как вектор признаков

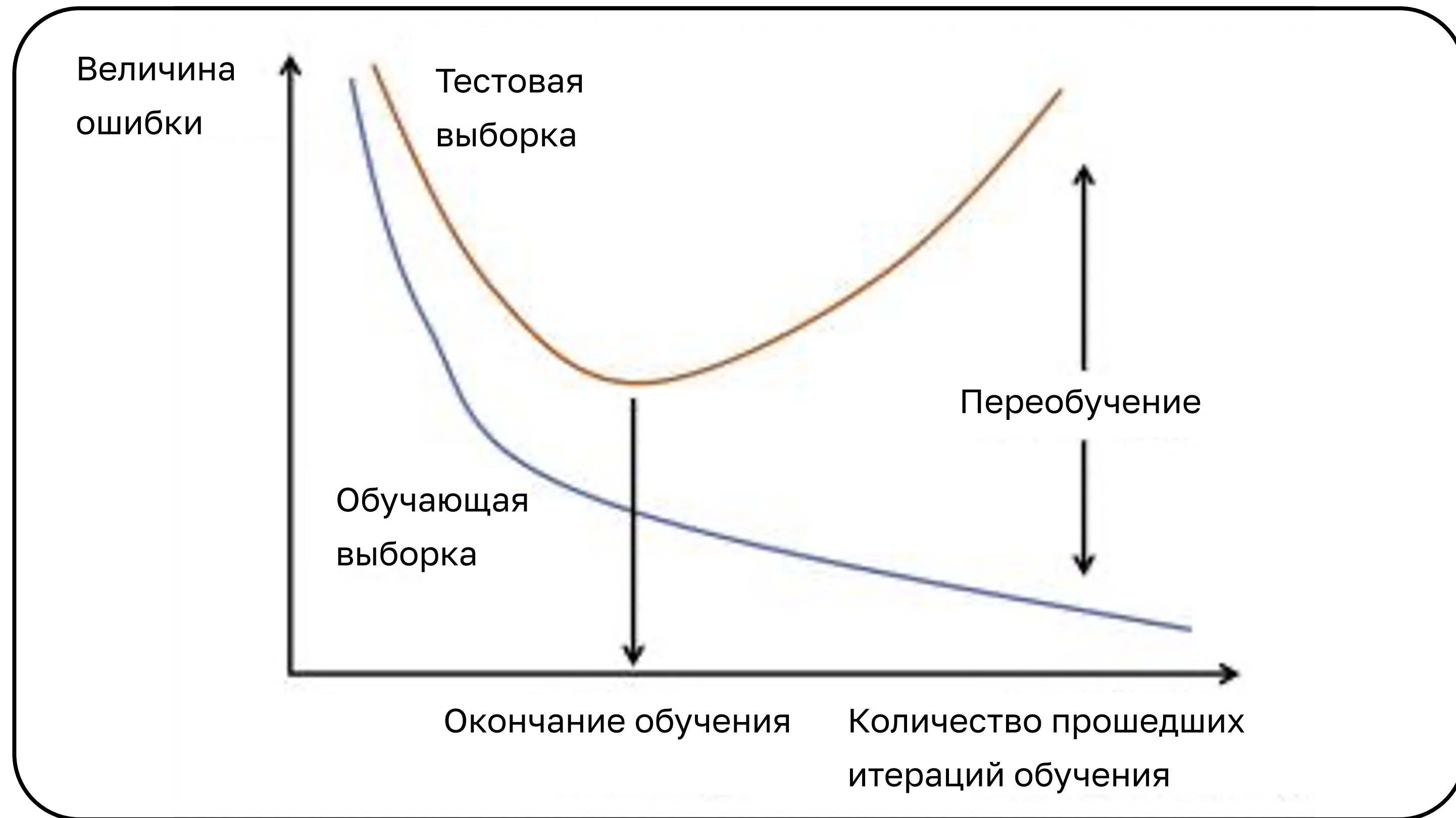
В зависимости от решаемой задачи нейронная сеть будет генерировать соответствующий ответ

Нейронные сети. Обработка изображений



Нейронные сети. Переобучение

Переобучение — это чрезмерное подстраивание под данные обучающей выборки, при котором ухудшается качество работы модели

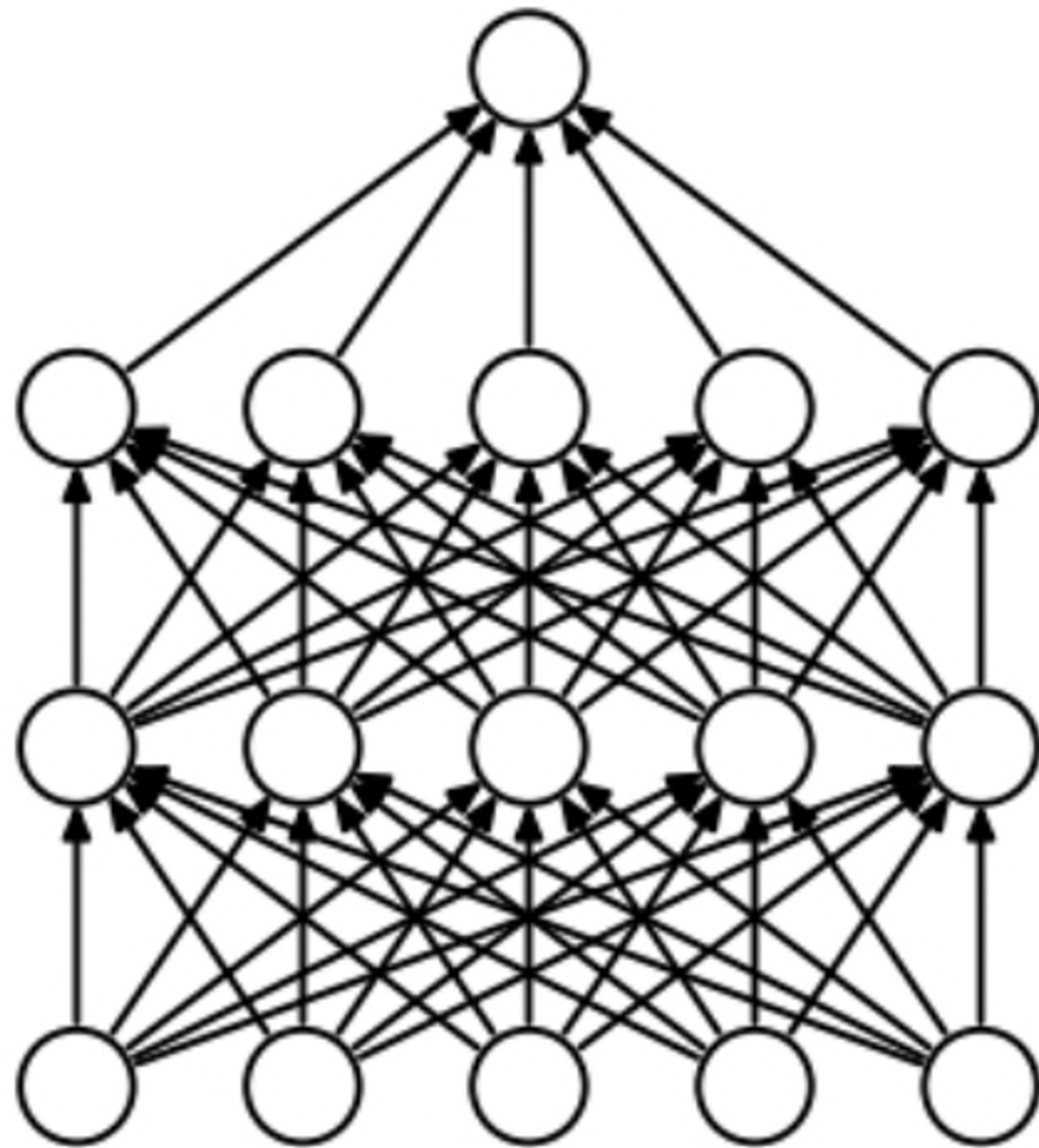


Нейронные сети. Dropout

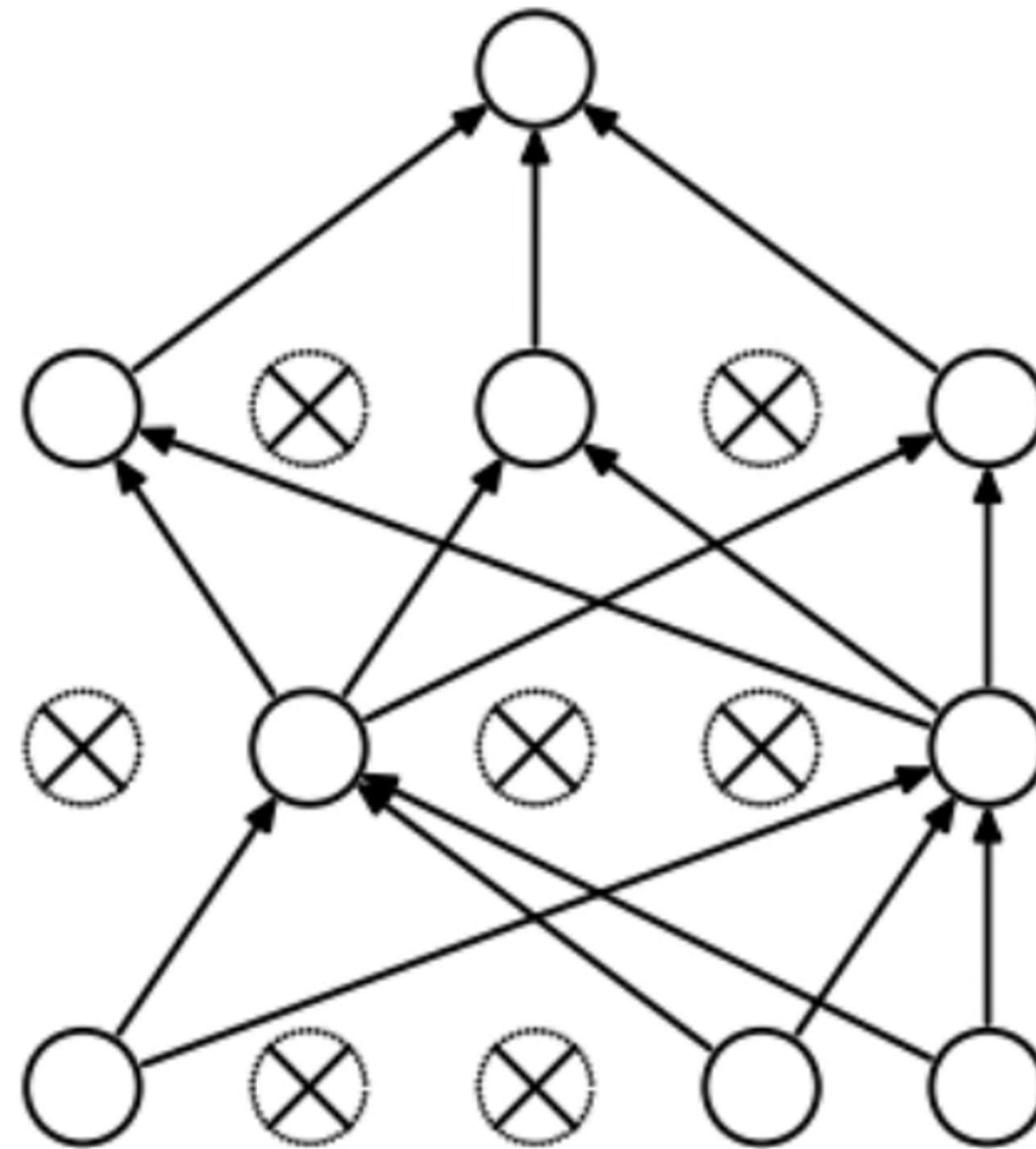
Dropout — это техника случайного обнуления заданного количества весов с тем, чтобы увеличить стабильность работы нейросети

Нейронные сети. Dropout

Dropout применяется **только** на стадии обучения модели. Если модель уже обучена, Dropout применять не нужно



(a) Стандартная нейронная сеть



(b) После применения dropout

Нейронные сети. Batch normalization

Batch normalization – это алгоритм, предназначенный для борьбы с затуханием/взрывом градиентов

Нейронные сети. Batch normalization

Также batch normalization ускоряет процесс обучения. Данные на каждом слое нормируются, чтобы их среднее и стандартное отклонения стали равны 0 и 1 соответственно.

При таком подходе этот слой содержит как обучаемые параметры, так и те, которые считаются с помощью усреднения по всей выборке

$$\mu^{[l]} = \frac{1}{m} \sum_{i=1}^m z^{[l](i)} \quad \sigma^{[l]} = \frac{1}{m} \sum_{i=1}^m (z^{[l](i)} - \mu^{[l]})$$

$$z_{norm}^{[l]} = \frac{z^{[l]} - \mu^{[l]}}{\sqrt{\sigma^2 + \varepsilon}} \quad \hat{z}^{[l]} = \gamma^{[l]} z_{norm}^{[l]} + \beta^{[l]}$$

Нейронные сети: резюме

- Нейронные сети – это обучаемые функции, причём они являются универсальными аппроксиматорами (могут приблизить любую непрерывную функцию)
- Нейронные сети склонны к переобучению: чрезмерному подстраиванию под обучающую выборку
- Чтобы избежать переобучения, применяют техники dropout и batch normalization

Практика



Практика

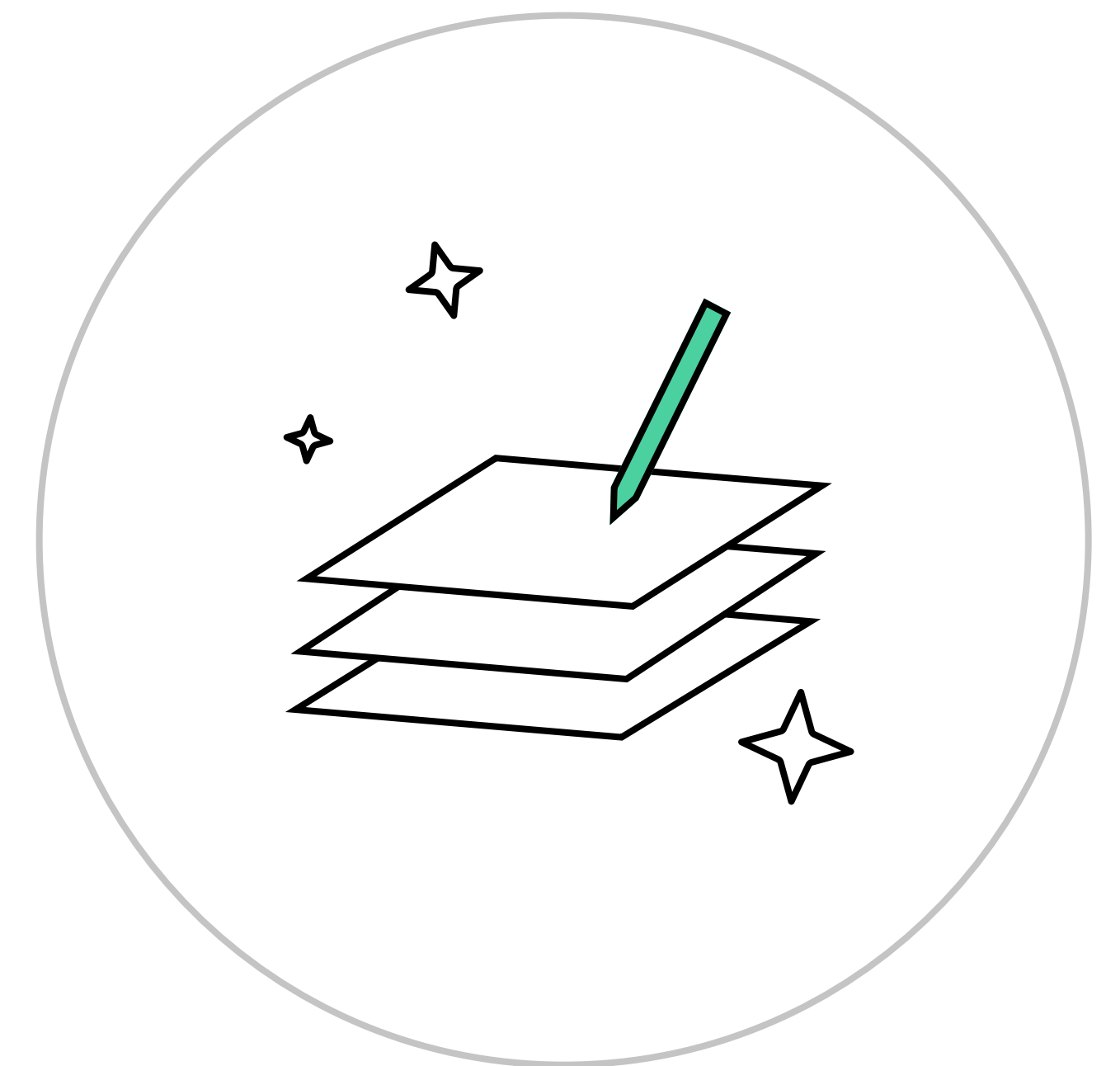
Цель: познакомиться с алгоритмами обработки изображений: PCA, мэтчинг характерных точек и гистограммой ориентированных градиентов (HOG).

Задача: построить классификатор изображений на основе метода HOG и обучить его.

Результат: обученный классификатор.

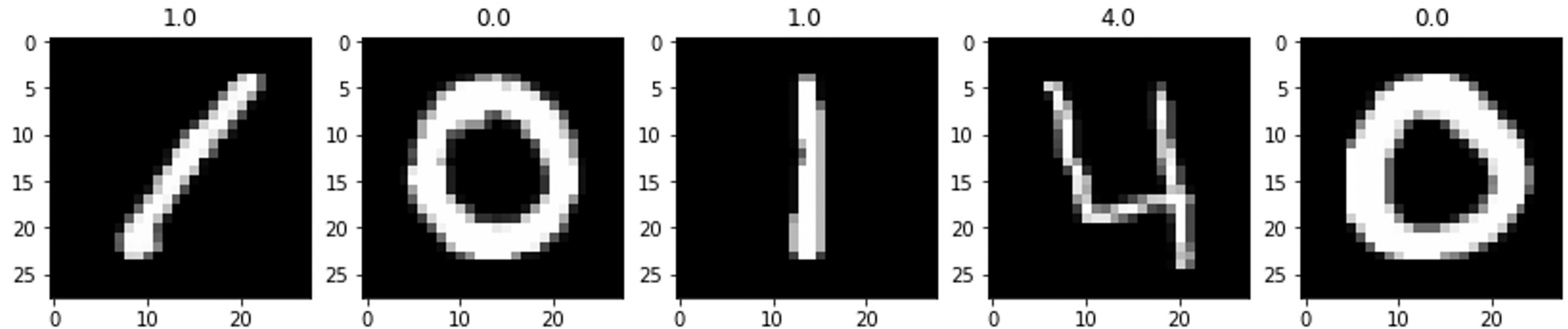
Инструменты:

- Python
- библиотеки NumPy, OpenCV



Практика

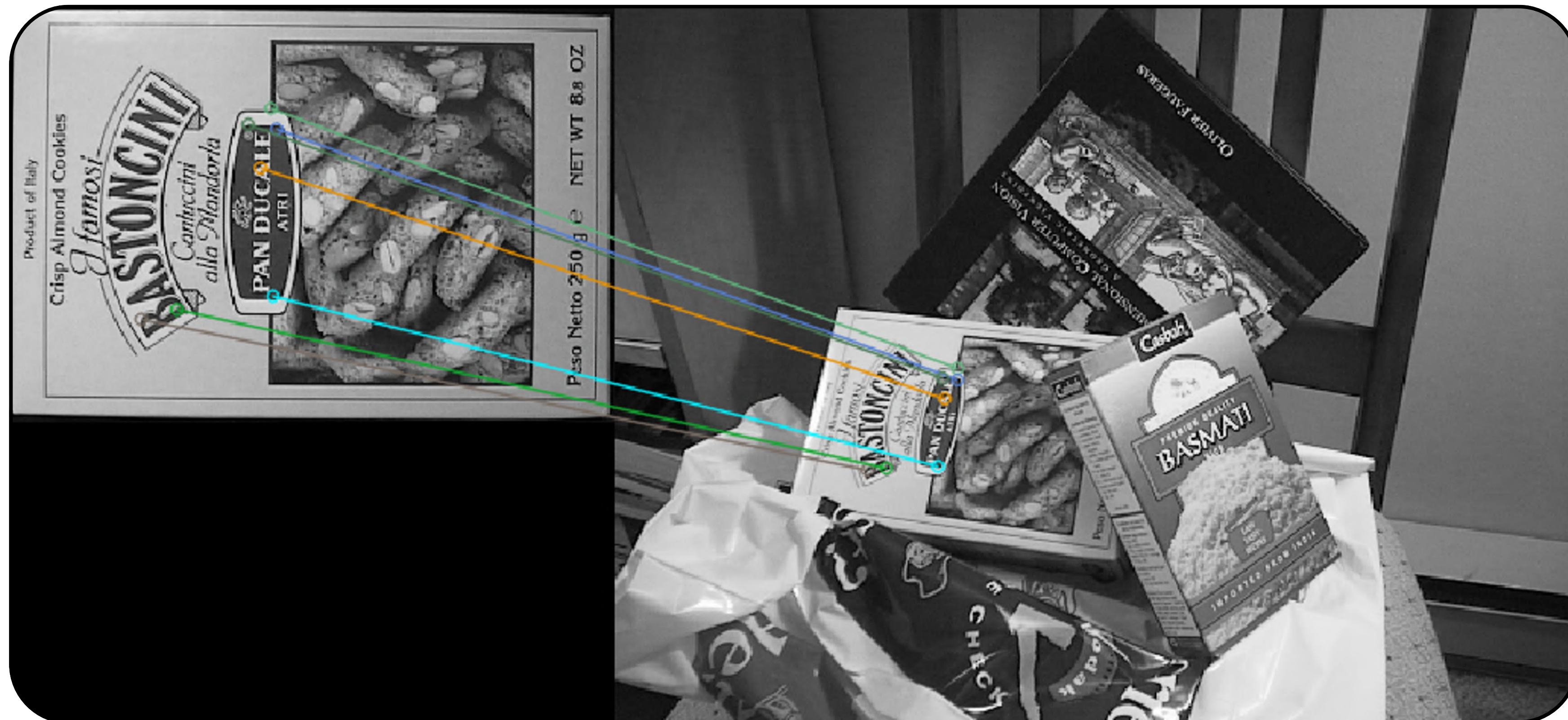
Распознавание рукописных цифр (датасет MNIST) с помощью метода НОГ



Практика

Учимся делать мэтчинг изображения по характерным точкам. Для этого:

- находим характерные точки на обоих изображениях и строим дескрипторы
- выбираем алгоритм для мэтчинга и проводим мэтчинг



Практика

Метод главных компонент (PCA):

- центрируем матрицу объекты-признаки
- проводим матричное разложение полученной матрицы
- восстанавливаем исходные объекты из пространства пониженной размерности

```
In [72]: # центрируем данные  
X_mean = X.mean(axis=0)  
X -= X_mean
```

```
In [73]: # матрица ковариации признаков  
cov = np.dot(X.T, X) / X.shape[0]
```

```
In [74]: U, S, _ = np.linalg.svd(cov)  
# U - собственные вектора матрицы ковариации  
# S - собственные значения
```


Итоги



Итоги

- 1 В результате PCA-преобразования можно получить сжатое представление изображения. Его удобно использовать для распознавания
- 2 Гистограммы цветов и градиентов более устойчивы к изменению цвета и поворотам, хорошо подходят для поиска визуально похожих изображений
- 3 Для мэтчинга изображений используют характерные точки



Итоги

- ④ Характерные точки выделяются большими значениями автокорреляционной матрицы
- ⑤ Нейронные сети позволяют приблизить любые непрерывные функции
- ⑥ Нейронные сети склонны к переобучению, поэтому необходимо внедрять методы dropout и batch normalization

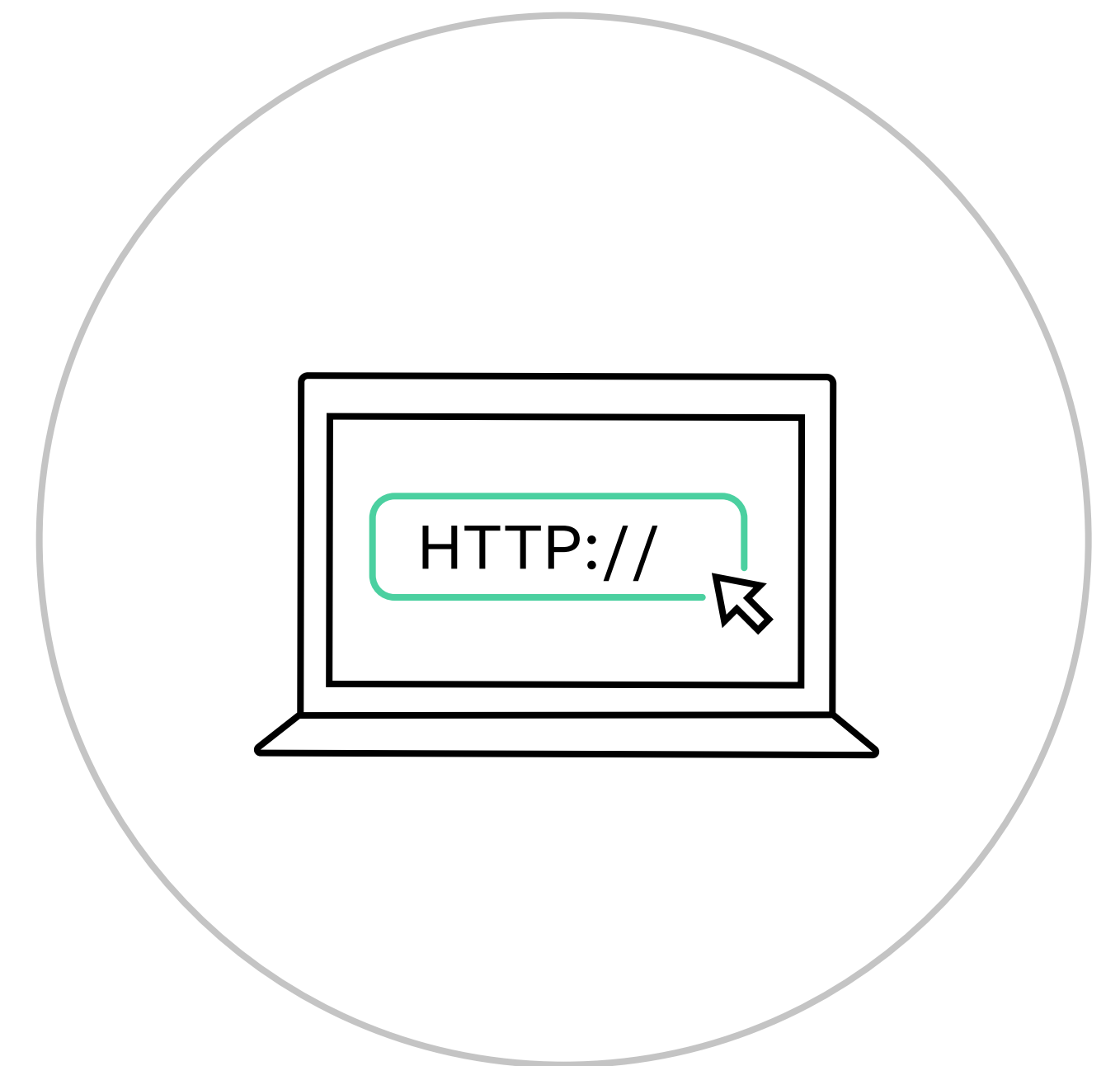


Дополнительные материалы



Дополнительные материалы к занятию

- Computer Vision: Algorithms and Applications (Chapter 4)
- OpenCV: Feature Detection and Description
- OpenCV-Python Tutorials
- Repository for OpenCV's extra modules



Выделение признаков и поиск похожих изображений

Егор Конягин

Сотрудник лаборатории высокопроизводительных
вычислений и больших данных Сколтеха

